Summer Internship Report

On

Bare-Metal Programming and Linux Porting on Zynq-7000 SoC Architecture for Particle Accelerator Control

At

Accelerator Instrumentation Section of Computer Control Division

Variable Energy Cyclotron Centre

1/AF, Bidhannagar, Kolkata - 700064

In partial fulfilment of the degree

Bachelor of Technology

In

Electronics & Telecommunications Engineering

Indian Institute of Engineering Science and Technology, Shibpur

Submitted By
SHASHANK DURLABH ARYA
2022ETB073

RIDDHIMAN BANERJEE
2022ETB067

Under the Guidance of

SHANTONU SAHOO (SCIENTIFIC OFFICER F)

TABLE OF CONTENTS

CHAPTER-IV:

Bare-Metal Programming

4.1 Introduction to Bare-Metal Development

4.2 Vivado Workflow

4.2.1 Creating a new block design and adding Zynq Processing System IP

4.2.2 Configure PS interfaces (UART, GPIO, AXI, MIO/EMIO) (if required)

4.2.3 Add peripherals (like AXI GPIO) using IP Integrator and configure them accordingly

4.2.4 Write a Constraint file (.xdc file) (if required)

4.2.5 Validate the Block Design, Create HDL wrapper and Generate Output Products

4.2.6 Generate Bitstream

4.2.7 Export the hardware platform (.xsa file) to be used in Vitis

4.3 Vitis Workflow

4.3.1 Create a new Application Project from .xsa file (exported from Vivado)

4.3.2 Choose a bare-metal template (e.g. “Hello World”)

4.3.3 Modify C code, for the target peripheral, according to the project

4.3.4 Build the project and run on the hardware

4.4 UART “Hello World” On ZedBoard

4.4.1 Description

4.4.2 Code

4.5 GPIO Via MIO (PS GPIO)

4.5.1 Description

4.5.2 Code

4.5.3 Results

4.6 AXI GPIO - Polling Implementation

4.6.1 Description

4.6.2 Code

4.6.3 Results

4.7 AXI GPIO – Interrupt-based Implementation

4.7.1 Description

4.7.2 Code

4.7.3 Results

4.7.4 Observations

CHAPTER-V:

Petalinux Porting on Zynq-7000 Soc

5.1 Introduction to Embedded Linux and Petalinux

5.2 Petalinux Installation

5.3 Hardware Description Import from Vivado

5.4 Creating and Configuring Petalinux Project

5.5 [Optional] Customizing Device Tree and Root Filesystem

ABSTRACT

This internship project, conducted at the Variable Energy Cyclotron Centre (VECC), focused on the implementation and exploration of embedded Linux systems and bare-metal programming on Xilinx Zynq-7000 System on Chip (SoC) architecture, with practical deployment and experimentation on ZedBoard and Zybo Z7 development platforms. The Zynq-7000 family integrates a dual-core ARM Cortex-A9 processing system (PS) with programmable logic (PL) on a single chip, offering a hybrid platform for both software and hardware co-design. The primary goal of this internship was to port PetaLinux onto a custom Zynq-based board and concurrently gain hands-on experience in bare-metal application development, emphasizing system-level understanding and peripheral interfacing.

The initial phase of the project involved the configuration and setup of a virtualized development environment using Ubuntu 20.04 LTS. Within this environment, Xilinx PetaLinux tools were installed and configured. A complete PetaLinux project was built by importing hardware definitions exported from Xilinx Vivado, and essential kernel, bootloader, and device tree configurations were made. Following the successful build and packaging of the project, the boot image (BOOT.BIN) and the kernel image (image.ub) were transferred onto an SD card, allowing the ZedBoard to boot into an embedded Linux environment. This exercise offered critical insights into the embedded Linux boot process, kernel packaging, and board-specific configuration essential for system-level development in real-world SoC applications.

In parallel, the second phase of the internship emphasized low-level bare-metal programming on the ZedBoard and Zybo Z7 platforms using the Xilinx Vivado Design Suite and Vitis IDE. A series of structured exercises were carried out, starting with a simple UART-based “Hello World” application. Subsequently, different modes of GPIO interfacing were explored: direct GPIO control through MIO and EMIO channels, and more sophisticated peripheral control using AXI_GPIO IP blocks in polling and interrupt modes. Interrupt-based GPIO interfacing was implemented to detect changes in programmable logic (PL) switches, triggering interrupts in the processing system (PS) and invoking an Interrupt Service Routine (ISR) that handled event-driven logic asynchronously.

ACKNOWLEDGEMENTS

We would like to extend my sincere gratitude to Siddharth Sir and Shantonu Sir, whose expert guidance, constant support, and valuable technical insights were pivotal to the successful completion of this internship project. Their mentorship not only enhanced my understanding of the Zynq-7000 SoC architecture and embedded Linux systems but also encouraged a deeper curiosity in system-level design and practical problem-solving. Their willingness to share knowledge and offer help at every step significantly enriched our learning experience. The collaborative environment at the Variable Energy Cyclotron Centre (VECC) provided us with a unique opportunity to work on real-world challenges and gain practical skills that go beyond academic learning. This experience has had a lasting impact on our technical growth, and We am truly grateful to both mentors for their support, patience, and encouragement throughout the duration of the internship.

CHAPTER-I

Introduction

This chapter provides a detailed overview of the foundational concepts, motivations, and tools used in the execution of this internship project. The work is centered around embedded system development and real-time control using the Xilinx Zynq-7000 SoC platform. This section also provides a brief description of the system components used in this project.

1.1 Motivation

Modern scientific instruments such as particle accelerators require fast, reliable, and configurable control systems to manage operations like timing, data acquisition, and feedback control. Traditional control architectures based on discrete microcontrollers or FPGAs often suffer from limitations in integration, latency, and flexibility. The Zynq-7000 SoC, which integrates a processing system (PS) with programmable logic (PL), offers a unified platform to build high-performance embedded control systems. This project aimed to explore the Zynq platform for real-world embedded applications and develop expertise in system booting, hardware-software co-design, and peripheral interfacing.

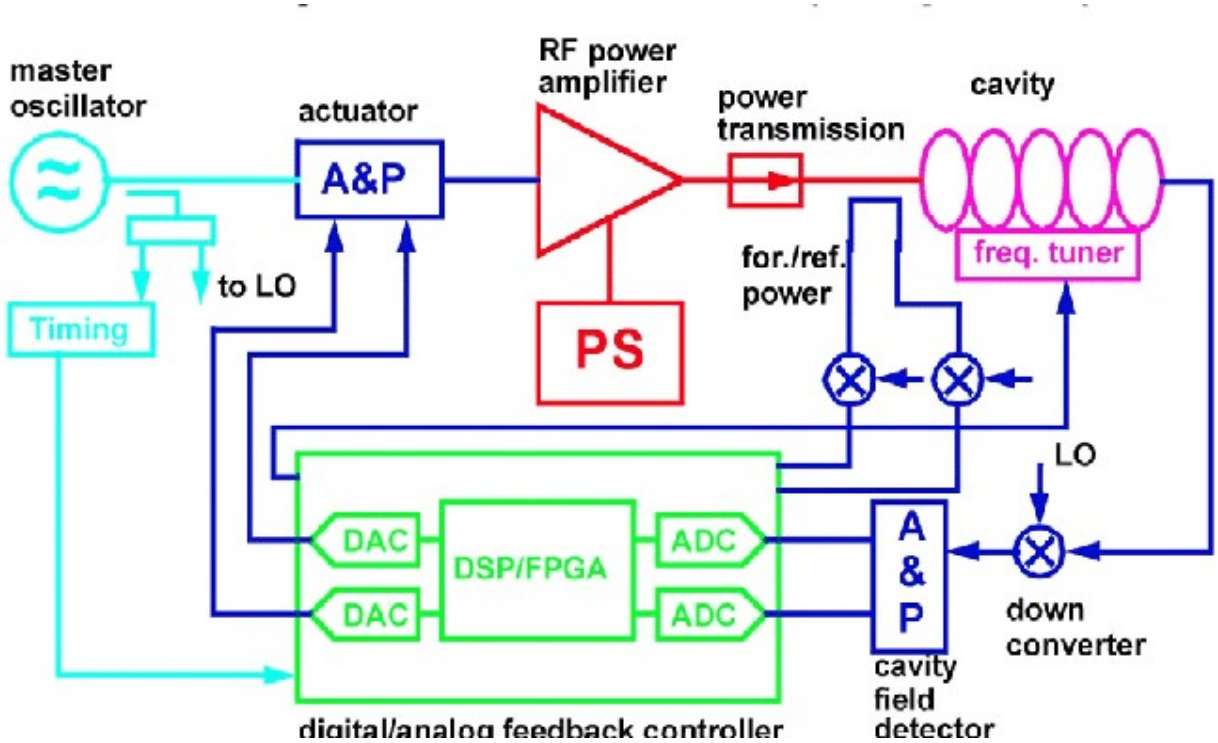
1.2 Particle Accelerator

A particle accelerator is a complex machine that accelerates charged particles, such as protons or electrons, to high speeds using electromagnetic fields. These machines require precise control over parameters such as beam position, timing, and energy levels. The control systems in accelerators are distributed and real-time in nature, often built using embedded systems for fast response. Components like LLRF (Low-Level RF) control, beam diagnostics, and feedback loops must operate with minimal latency and high reliability, demanding robust embedded platforms.

1.3 Embedded Control Systems

Embedded control systems are dedicated computing systems designed to perform specific control tasks within larger systems. In the context of a particle accelerator, these systems handle controlling RF generators, motorized components, and data acquisition units. Such systems must be real-time, deterministic, and robust against noise and interference. The Zynq-7000 SoC is an excellent fit for such applications due to its ability to implement both software and custom hardware logic on a single chip.

1.4 LLRF Control



Low-Level Radio Frequency (LLRF) control is a crucial subsystem in a particle accelerator. It ensures the stability and synchronization of the RF fields used to accelerate particles. LLRF systems must perform high-speed signal processing, real-time feedback, and adaptive control, typically requiring FPGA-like reconfigurability and processor-like flexibility. A Zynq-7000 SoC allows LLRF systems to be implemented with tight integration between the control logic in the FPGA (PL) and algorithms running on the ARM processor (PS).

1.5 Why Zynq-7000 SoC Over independent FPGAs And Microcontrollers?

The Zynq-7000 SoC combines the capabilities of an ARM Cortex-A9 processor and Xilinx FPGA fabric within a single chip. This eliminates the need for separate microcontroller and FPGA devices, thus reducing board space, interconnect complexity, and latency. The shared memory and high-speed AXI interface between the PS and PL facilitate faster communication and seamless data transfer. It allows hardware accelerators in PL to be tightly coupled with software tasks in PS, enabling high-performance applications.

1.6 Why PetaLinux for Embedded Linux Development?

PetaLinux is a Xilinx-supported embedded Linux porting tool specifically designed for Zynq and other Xilinx platforms. It allows developers to build customized Linux systems, including kernel configurations, bootloaders, and device trees. PetaLinux simplifies the integration of custom hardware drivers and applications while providing access to rich set of open-source Linux utilities. Compared to bare-metal development, PetaLinux, an OS based design enables faster development cycles, better debugging tools, and easier management of complex software stacks, making it easier for control systems and prototyping.

1.7 System Components

To implement and test the concepts described above, the following tools and hardware platforms were used:

1.7.1 ZedBoard

The ZedBoard is a development platform based on the Zynq-7000 SoC. It features a dual-core ARM Cortex-A9 processor and Artix-7 programmable logic, making it ideal for exploring both software and hardware aspects of embedded system design. It includes peripherals such as LEDs, push buttons, switches, UART, Ethernet, and GPIO for rapid prototyping.

1.7.2 Xilinx Vivado

Vivado Design Suite is Xilinx's integrated environment for hardware design, IP integration, simulation, Implementation and bitstream generation. It enables developers to configure the programmable logic of the Zynq SoC and create hardware platforms that may or may not interact with the processing system. The Hardware Description File (HDF) or XSA file generated from Vivado is essential for PetaLinux and Vitis integration.

1.7.3 Xilinx Vitis

Vitis is Xilinx's unified software development environment for writing and deploying applications on the processing system of the Zynq SoC. It supports bare-metal as well as Linux-based development. In this project, Vitis was used for developing UART and GPIO control programs, both in polling and interrupt-driven modes, using the BSP generated from Vivado.

1.7.4 PetaLinux

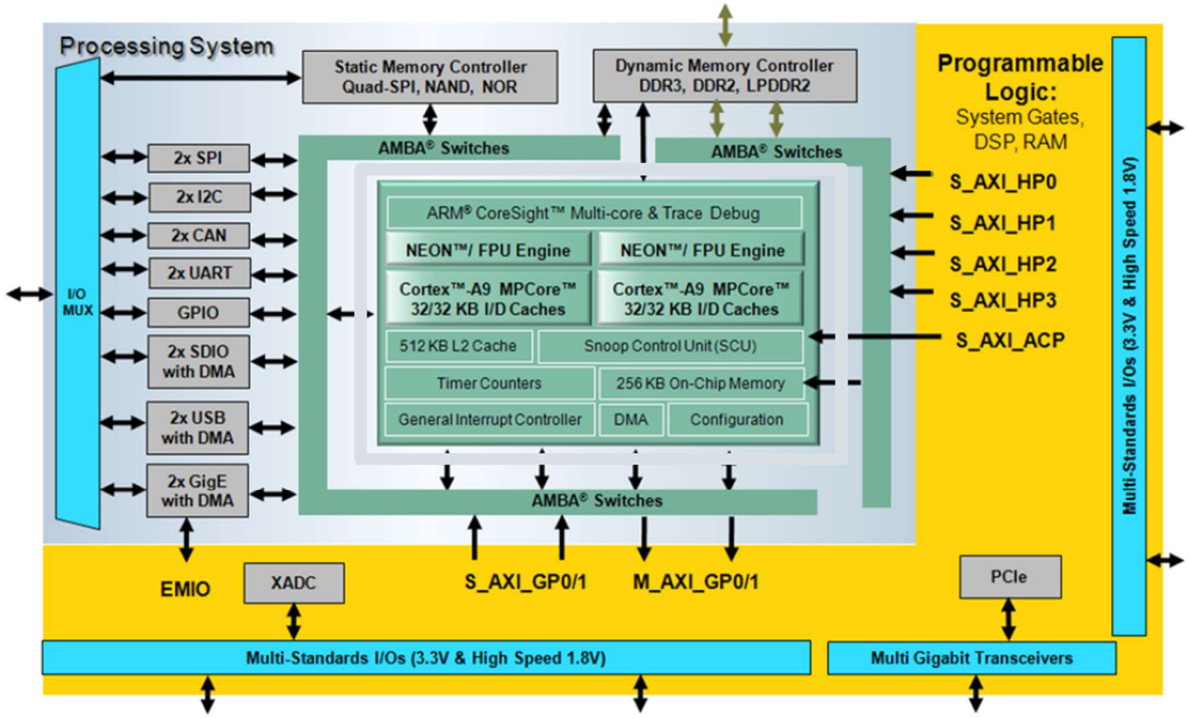
PetaLinux provides a build system for creating customized Linux distributions for Xilinx SoCs. It was used in this project to build and deploy an embedded Linux image for ZedBoard, including BOOT.BIN and image.ub. The integration of custom device trees and kernel modules with pre-built hardware platforms allowed successful booting of Linux on ZedBoard using SD card-based deployment.

CHAPTER-II

Hardware Overview

This chapter provides an overview of the core hardware technologies used in the project, namely Zynq-7000 SoC architecture. It also includes details of the ZedBoard and Zybo Z7 development boards, which were used for implementation and testing.

2.1 Zynq-7000 SoC Architecture



The Xilinx Zynq-7000 family of devices is built on the concept of tightly coupling a dual-core ARM Cortex-A9 Processing System (PS) with a 7-series Xilinx Programmable Logic (PL) fabric. This integration enables designers to implement complex systems with high performance, low latency, and efficient hardware-software partitioning. Parallel tasks such as data acquisition, control loops can be implemented on PL. It is especially suited for certain embedded control applications, digital signal processing, and real-time systems which requires parallel computation or acquisition.

2.1.1 Processing System (PS)

The Processing System (PS) is the general-purpose computing unit of the Zynq-7000 SoC. It includes:

- Dual-core ARM Cortex-A9 processors (running up to 1 GHz)
- 256 KB of on-chip memory (OCM)
- Level 1 and Level 2 caches
- Rich set of peripherals: UART, SPI, I2C, USB, SDIO, CAN, and Gigabit Ethernet
- DDR3/DDR2 memory controllers
- General-purpose I/O (MIO) and extended memory-mapped I/O (EMIO) interfaces

The PS operates independently of the PL and can boot standalone using conventional boot modes like QSPI Flash, SD card, or JTAG.

2.1.2 Programmable Logic (PL)

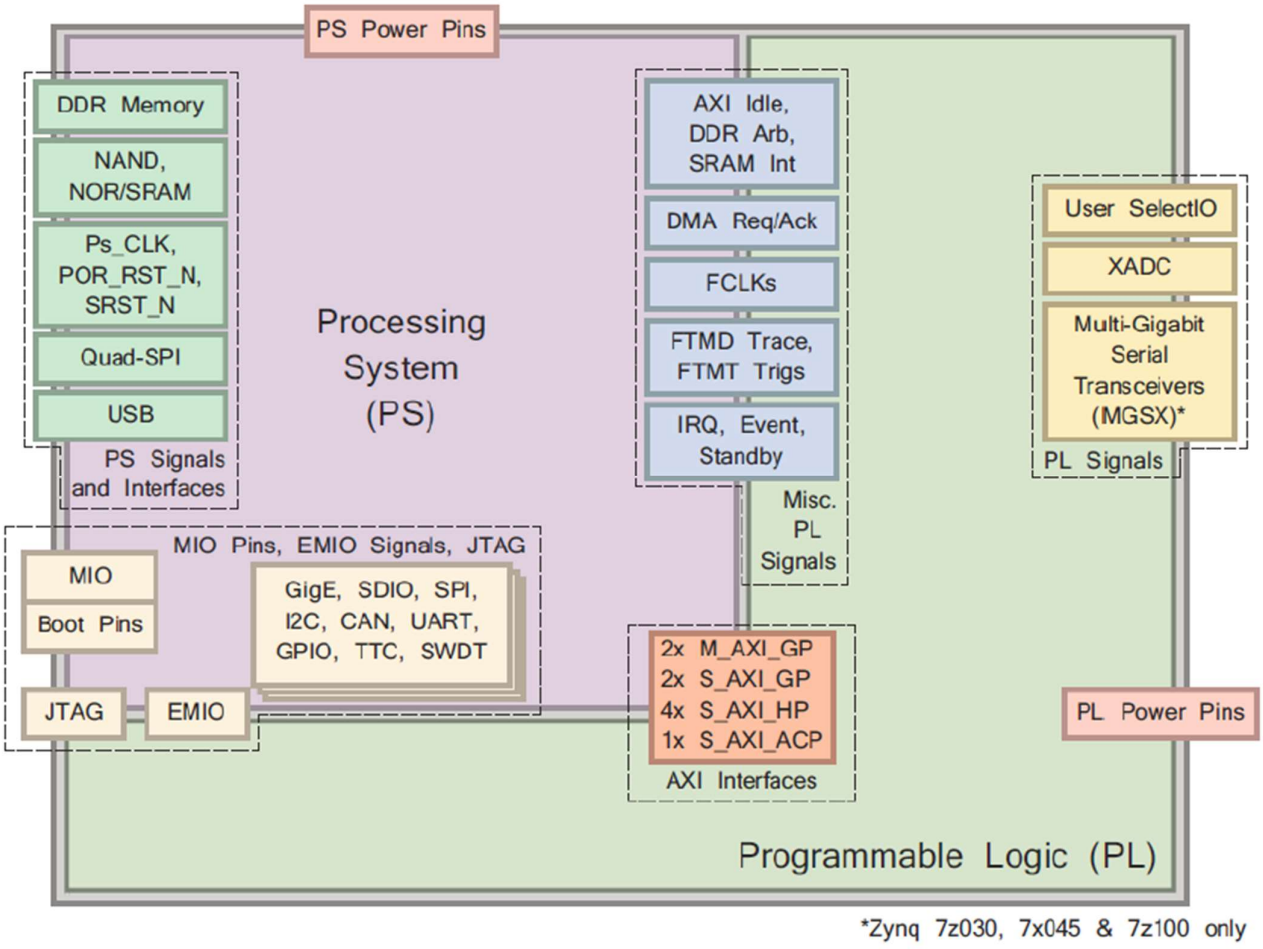
The Programmable Logic (PL) is derived from Xilinx's 7-Series FPGA fabric (Artix-7 or Kintex-7 depending on the device). It provides reconfigurable hardware for implementing custom logic, accelerators, signal processing pipelines, and interfaces to external peripherals.

Key components of PL include:

- Configurable Logic Blocks (CLBs) with LUTs and flip-flops
- Block RAM (BRAM) for internal storage
- Digital Signal Processing (DSP) slices for arithmetic operations
- Clock Management Tiles (CMT) for PLL/MMCM configurations
- General-purpose I/Os and high-speed serial transceivers

The PL can be programmed using VHDL/Verilog or through Vivado IP Integrator using pre-built Xilinx IP cores.

2.1.3 PS – PL Interface



An important feature of the Zynq architecture is the interconnection between PS and PL. These interfaces enable the two subsystems to communicate efficiently and share data for co-processing tasks.

Main PS-PL interfaces include:

- AXI General Purpose (GP) Ports: Bi-directional interfaces for software-accessible logic (e.g., AXI GPIO)
- EMIO (Extended MIO): Allows routing of peripheral signals from the PS through the PL
- AXI Accelerator Coherency Port (ACP): Enables coherent data sharing between the PL and PS cache

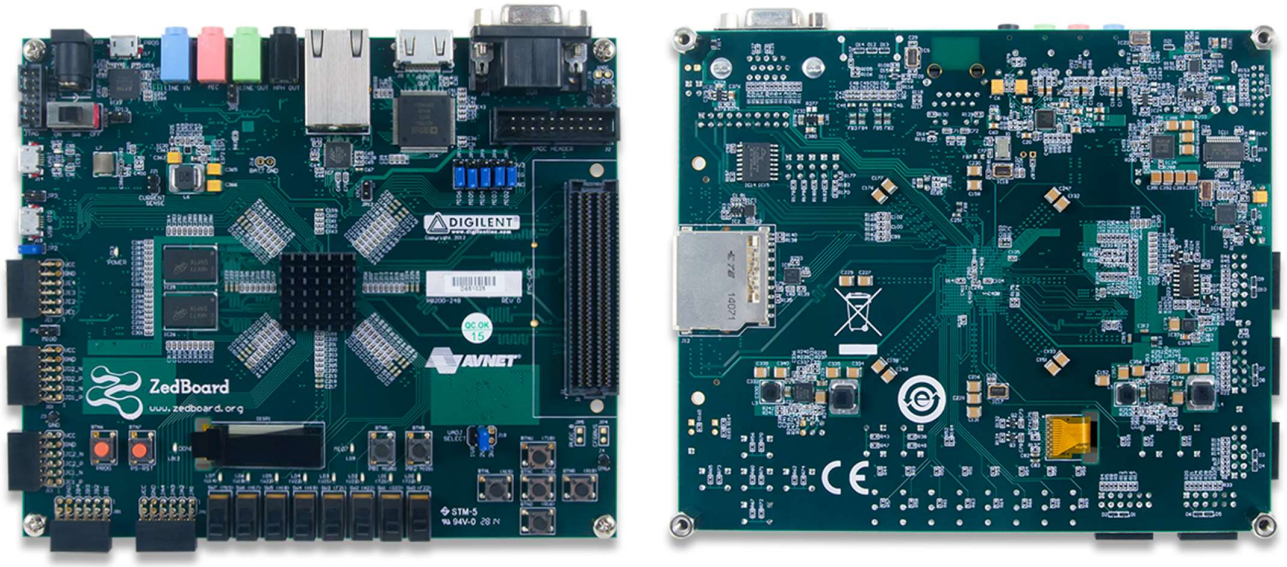
- AXI High Performance (HP) Ports: Unidirectional, high-throughput channels for large data transfers from PL to PS

These interfaces make it possible to offload computation-heavy tasks to hardware while maintaining software programmability and control flow.

2.2 Development Boards

The hardware implementation and testing during the internship were carried out using two popular Zynq-based development Boards: ZedBoard and Zybo Z7. Both boards offer extensive features to explore the Zynq-7000 platform but cater to slightly different use cases.

2.2.1 ZedBoard



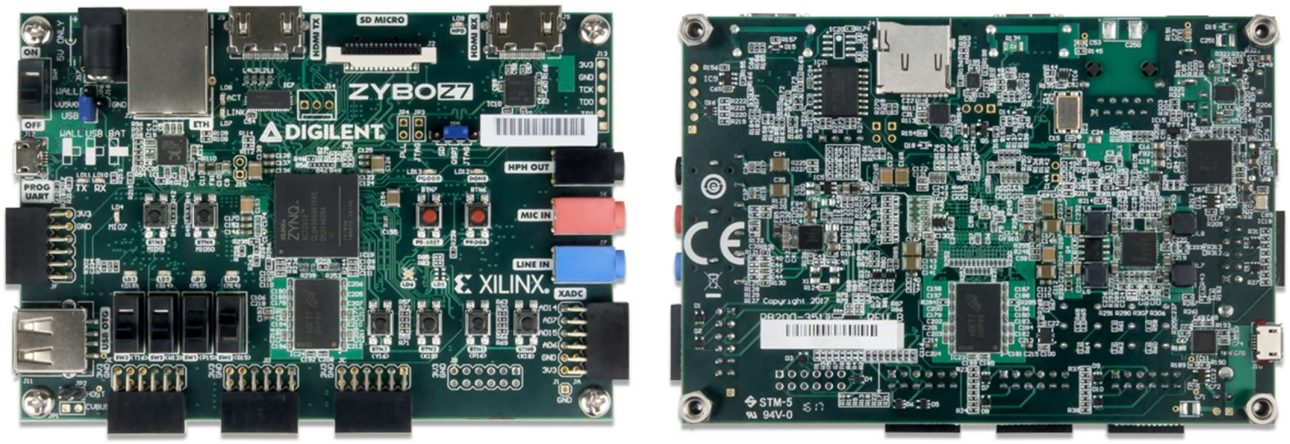
The ZedBoard is based on the **Zynq-7020** SoC and is designed for embedded system prototyping. It offers a rich set of peripherals and expansion options to support a wide range of applications.

Key specifications:

- Zynq-7020 SoC (Dual-core ARM Cortex-A9 + Artix-7 PL)
- 512 MB DDR3 RAM connected to the PS
- 256 MB Flash, USB OTG, Gigabit Ethernet, SD card slot
- HDMI output, USB UART
- User LEDs, switches, and buttons
- Multiple Pmod connectors for custom I/O

The ZedBoard was used in this project for PetaLinux deployment and bare-metal programming, including GPIO polling and interrupt handling.

2.2.2 Zybo Z7



The Zybo Z7 is a compact, cost-effective development board intended for academic and embedded vision applications. It is available in Zynq-7010 and Zynq-7020 variants.

Key specifications:

- Zynq-7010 or 7020 SoC
- 1 GB DDR3 RAM
- HDMI input and output
- USB OTG, USB UART, and 10/100 Ethernet
- 6 LEDs, 4 switches, 4 pushbuttons
- Pmod and Arduino-compatible headers

The Zybo Z7 was utilized for bare-metal application development, particularly for GPIO interfacing through MIO and EMIO. It provided a flexible platform for hardware-software co-design and testing real-time input/output operations.

CHAPTER-III

Tools and Development Environment Setup

This chapter outlines the comprehensive setup process for the tools and environments used during the internship project. Working with embedded platforms like the Zynq-7000 SoC and PetaLinux requires a carefully configured toolchain and robust development environment. The process involved both local and remote machine configurations, virtualization for Linux development, and tool installations essential for hardware-software co-design.

3.1 System Requirements

For this project, we require:

- Xilinx Vivado 2023.1 (64 bit) and Xilinx Vitis 2023.1 (64 bit)
- Linux OS (Ubuntu 20.04 LTS)
- Oracle VirtualBox (for running Ubuntu VM)
- Serial Terminal Emulator like minicom, PuTTY

3.2 Installation of Vivado and Vitis

Xilinx Vivado Design Suite 2023.1 and Vitis Unified IDE 2023.1 were installed to design the hardware platform and develop the embedded software respectively.

- Vivado was used for configuring the Zynq Processing System IP, connecting peripherals like AXI GPIO, and generating hardware description files (.xsa).
- **Vitis** was installed alongside Vivado, and it provided the software environment for developing C/C++ applications targeting the ARM cores.

Both tools were installed from the official Xilinx website. Appropriate environment variables were set to source the toolchains correctly in both local and remote environments.

3.3 Remote Server

Due to resource constraints on local system, a remote server was used for Vivado, Vitis developments and running large builds (e.g. PetaLinux). Access to the server was granted via remote desktop clients which will be explained in the next section.

3.4 Remmina

Remmina is a remote desktop client used for graphical access to the remote server. It supports RDP, VNC, SSH, and SPICE protocols. Remmina allowed efficient access to the server over an RDP session. To access the server over an RDP session, the server IP address, username, username's password and port number are needed.

3.5 Hardware Server on Local Machine

To perform hardware-related operations such as programming the ZedBoard or Zybo Z7, a local Vivado hardware server was launched. The ZedBoard is connected to host machine using USB cable, which provides a JTAG interface. Hardware server detects this JTAG interface and listens for JTAG programming commands. Vivado on remote machine sends commands over network to hardware server which drives the JTAG interface to program the board. This setup enabled us to program a local board from a remote Vivado or Vitis session. The ***hw_server*** file can be found in the ***bin*** directory of the Vivado installation. To just run a ***hw_server,*** we can just work with a lightweight version called Vivado Lab Edition rather than installing whole Vivado suite, however it doesn't fulfil the functions of synthesizing, implementing or editing the created designs.

3.6 Proxy Setup on Local Machine & Remote Server

Since internet access was routed through a proxy (in VECC network), proxy configuration was required.

Permanent Proxy settings were configured in ***.bashrc*** files:

```
export http_proxy=http://proxy.example.com:port
export https_proxy=https://proxy.example.com:port
```

The same settings were applied on the remote server for consistent network access.

3.7 Ubuntu 20.04 LTS In Virtual Box for Petalinux

The remote server had CentOS Linux release 7.9.2009 (Core). But PetaLinux was not supported on this particular version of CentOS, it is officially supported only on specific versions of Linux. Hence, a virtual machine was created using Oracle VirtualBox and Ubuntu 20.04 LTS was installed as the guest OS.

Issues that may arise during this section are detailed in Appendix C.1.

CHAPTER-IV

Bare-Metal Programming

This chapter focuses on the bare-metal programming aspect of the internship project. Bare-metal development refers to writing software that runs directly on hardware without the abstraction or services of an operating system. It provides developers with complete control over memory, peripherals, and execution flow - making it ideal for real-time and low-latency embedded applications. In this project, a variety of bare-metal programs were implemented on the ZedBoard and Zybo Z7 using the Xilinx Vivado and Vitis development tools.

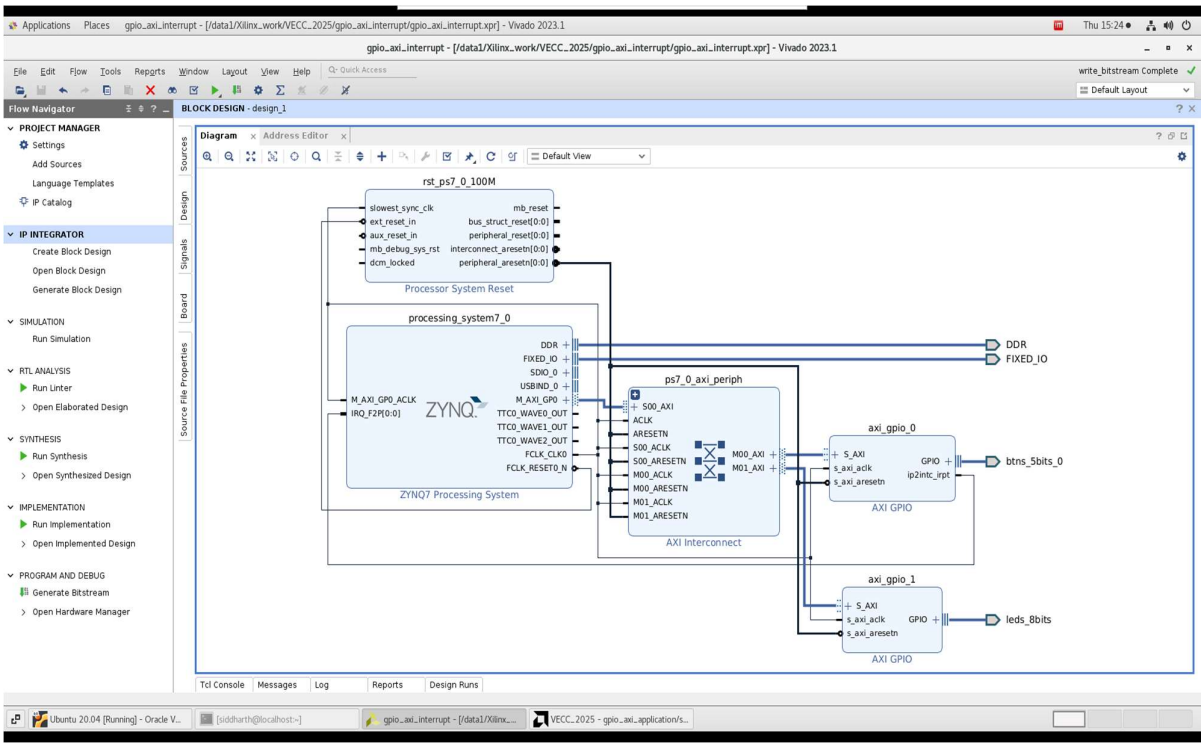
4.1 Introduction To Bare-Metal Development

Bare-metal programming involves creating applications that run directly on the processor core without relying on any OS-level features like scheduling, system calls, or memory protection. This mode of development is essential for real-time applications where deterministic timing and full hardware control are critical.

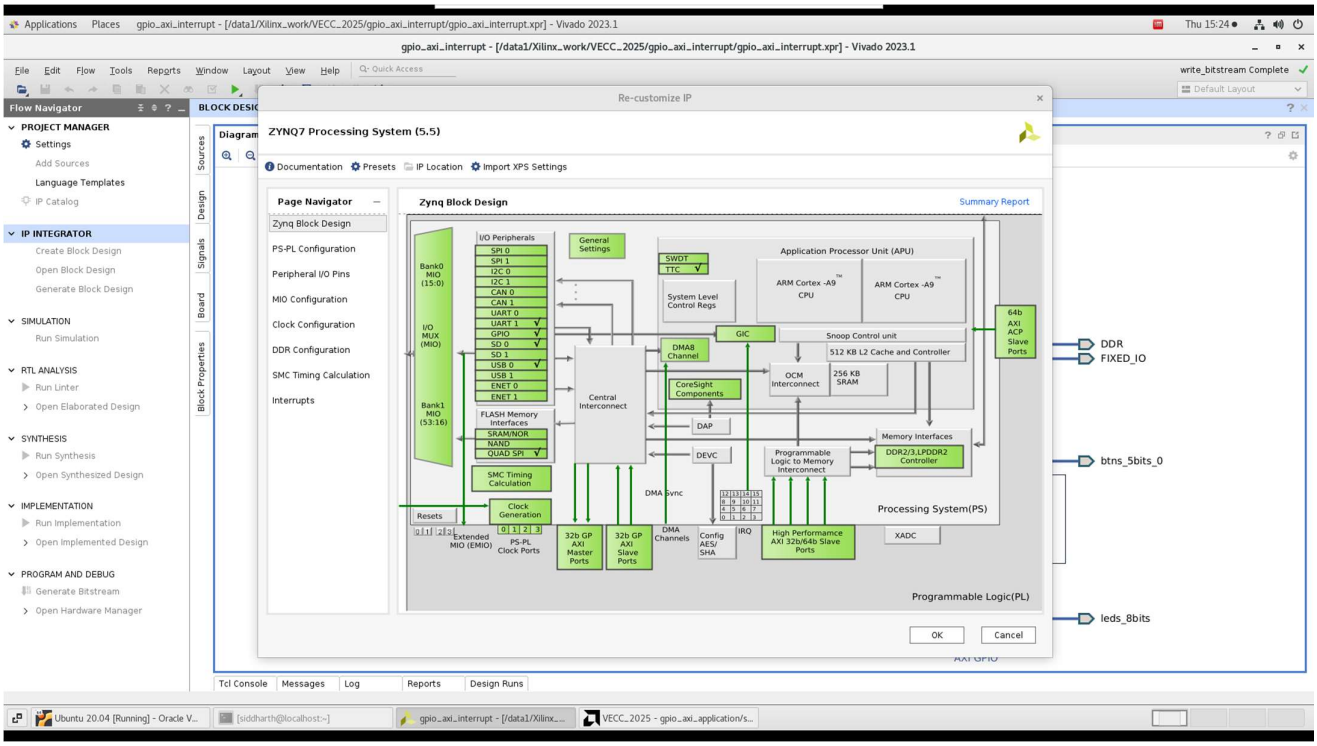
In the Zynq-7000 SoC, bare-metal code is typically deployed on the ARM Cortex-A9 processor by running application project, containing the imported .xsa file having description of custom hardware developed in Vivado.

4.2 Vivado Workflow

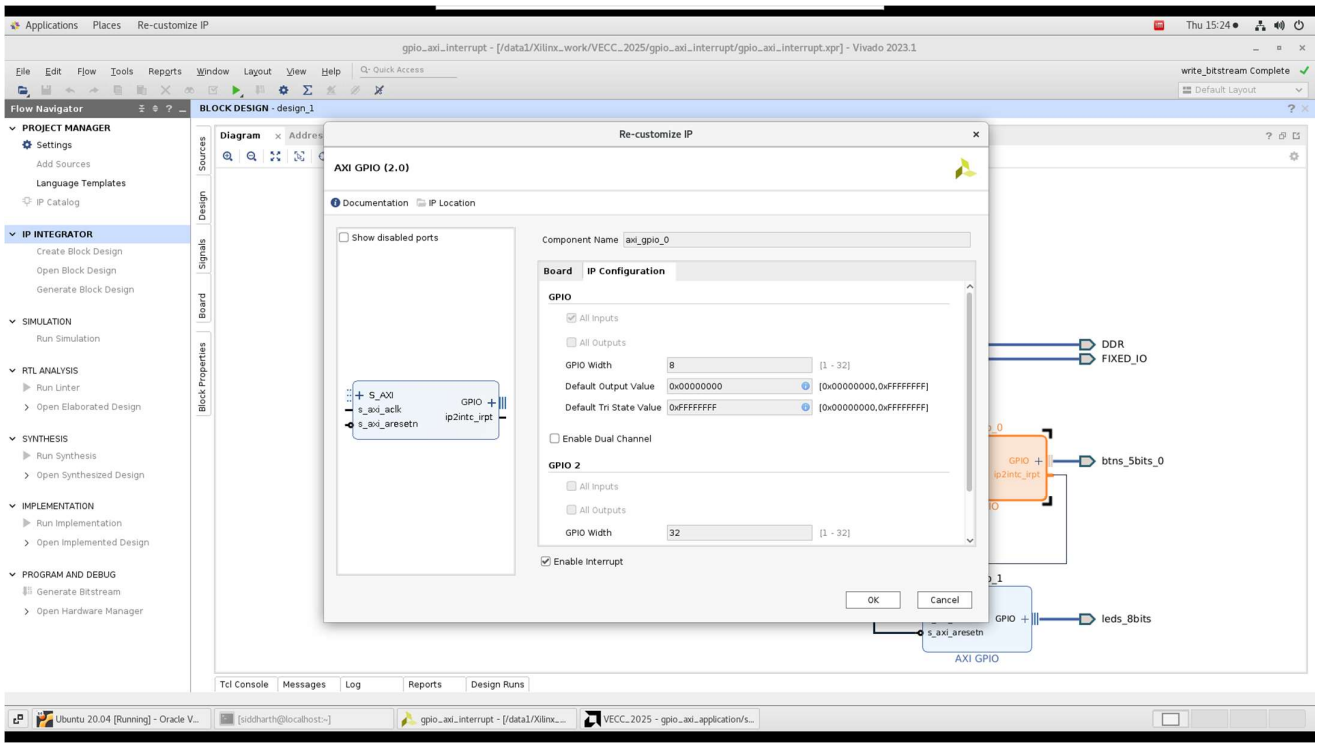
4.2.1 Creating a new block design and adding Zynq Processing System IP



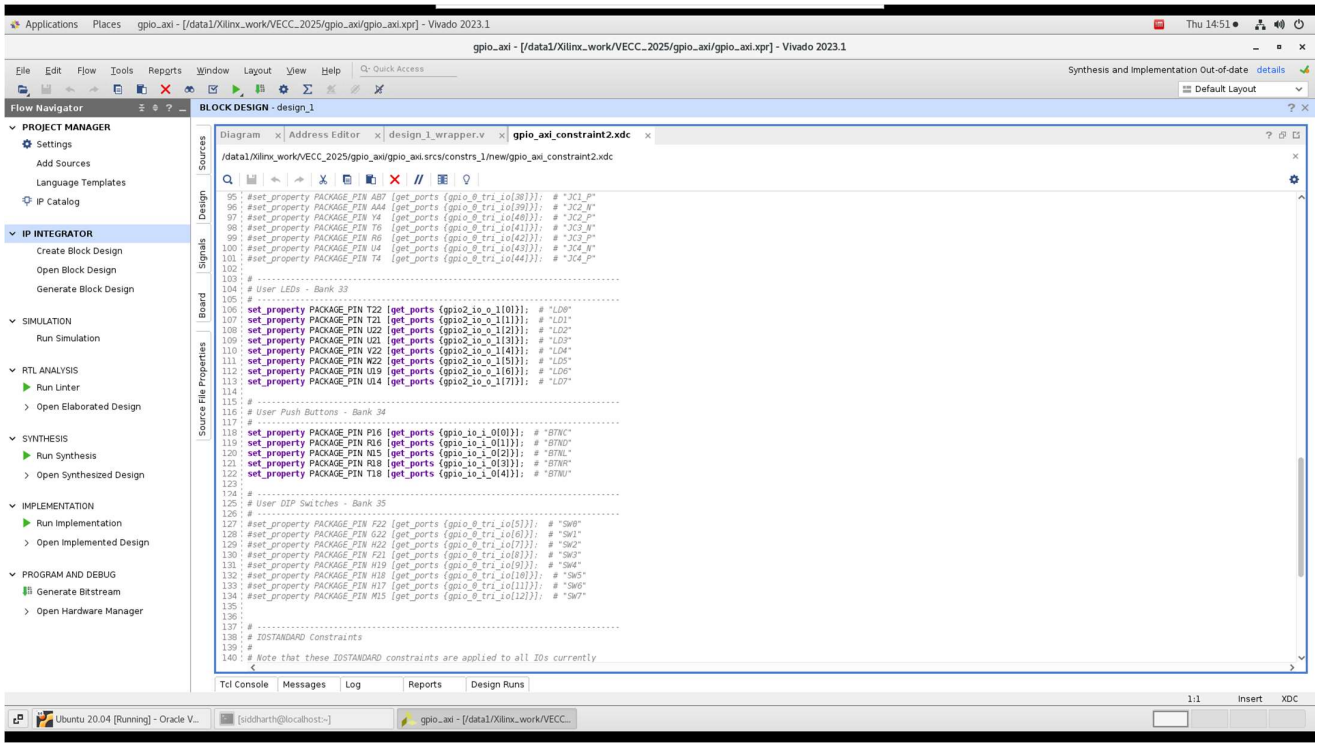
4.2.2 Configure PS interfaces (UART, GPIO, AXI, MIO/EMIO) (if required)



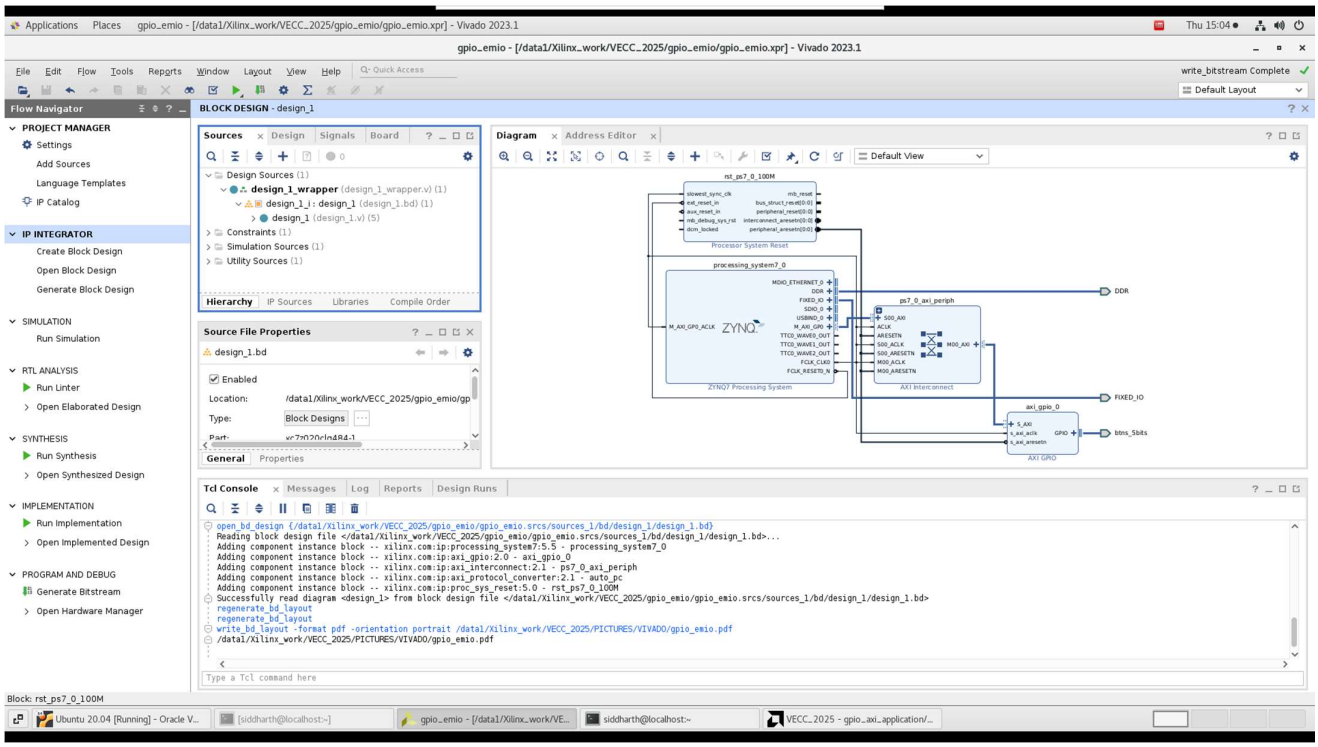
4.2.3 Add peripherals (like AXI GPIO) using IP Integrator and configure them accordingly



4.2.4 Write a Constraint file (*.xdc* file) (if required)



4.2.5 Validate the Block Design, Create HDL wrapper and Generate Output Products



4.2.6 After successful synthesis and implementation, Generate Bitstream

4.2.7 Export the hardware platform (*.xsa* file) to be used in Vitis

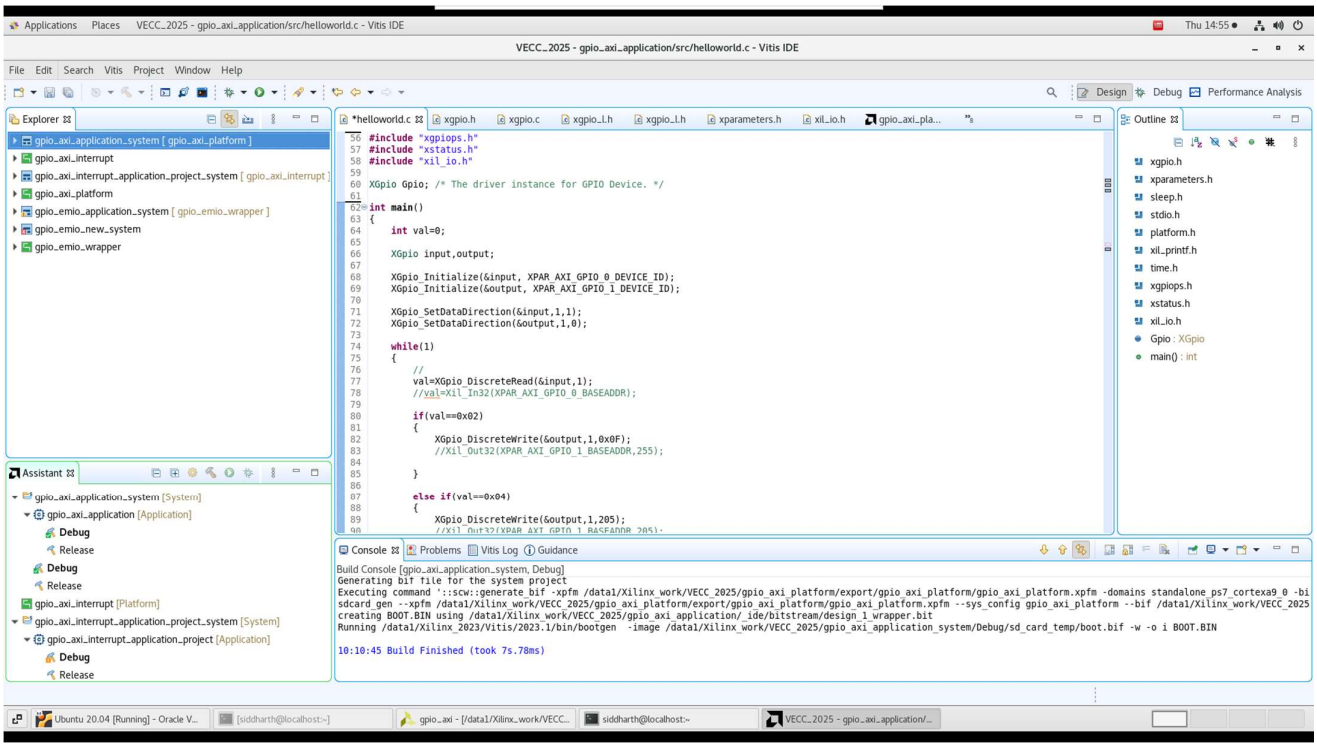
[Optional] Compress Bitstream

4.3 Vitis Workflow

4.3.1 Create a new Application Project including *.xsa* file (exported from Vivado)

4.3.2 Choose a bare-metal template (e.g. “Hello World”)

4.3.3 Modify C code, for the target peripheral, according to the project



4.3.4 Build the project and run on hardware

Issues that may arise during this section are detailed in Appendix A.2.

4.4 UART “Hello World” On Zedboard

4.4.1 Description

The program would display “Hello World” on serial terminal using UART. Two cables were connected to ZedBoard, one for programming and other for UART communication. The result was verified on Vitis emulator as well as PuTTY after setting proper configuration for serial communication such as baud rate, parity and stop bits.

Issues that may arise during this section are detailed in Appendix A.1.

4.4.2 Code

```
#include <stdio.h>
#include "platform.h"
int main() {
        init_platform();
        printf("Hello World\n\r");
        cleanup_platform();
        return 0;
```

}

4.5 GPIO Via MIO (PS GPIO)

4.5.1 Description

The Vivado and Vitis workflow is same as described above. The bare-metal C program would read input from PS GPIO pin 50 (push button) and write input data to PS GPIO pin 7 (built-in LED). Additionally, The program would control Pmod pins according to the input data (the input read from PS GPIO pin 50).
Note: Here everything is done using processor as well as memory mapped I/Os (MMIO). There is no involvement of PL section in this case.

4.5.2 Code

```
#include "xgpio.h"
#include "xparameters.h"
#include "xil_printf.h"
#include "sleep.h"

#define BTN_GPIO_ID XPAR_GPIO_0_DEVICE_ID
#define LED_GPIO_ID XPAR_GPIO_1_DEVICE_ID
#define PMOD_GPIO_ID XPAR_GPIO_2_DEVICE_ID

#define CHANNEL 1
int main() {

	XGpio btnGpio, ledGpio, pmodGpio;
	int btnState;

	// Initialize all GPIOs
	XGpio_Initialize(&btnGpio, BTN_GPIO_ID);
	XGpio_Initialize(&ledGpio, LED_GPIO_ID);
	XGpio_Initialize(&pmodGpio, PMOD_GPIO_ID);

	// Set directions
	XGpio_SetDataDirection(&btnGpio, CHANNEL, 0xFF);  // Input
	XGpio_SetDataDirection(&ledGpio, CHANNEL, 0x00);  // Output
	XGpio_SetDataDirection(&pmodGpio, CHANNEL, 0x00); // Output

	xil_printf("Starting GPIO test\n\r");

	while (1) {
		btnState = XGpio_DiscreteRead(&btnGpio, CHANNEL);
```

```
        if (btnState & 0x01)
        {
                // Button pressed
                // Turn LED ON
                XGpio_DiscreteWrite(&ledGpio, CHANNEL, 1);
                // Write to Pmod
                XGpio_DiscreteWrite(&pmodGpio, CHANNEL, 1);
                usleep(10000);
                XGpio_DiscreteWrite(&pmodGpio, CHANNEL, 0);
        }

        else
        {
                // Button not pressed
                XGpio_DiscreteWrite(&ledGpio, CHANNEL, 0);   // Turn OFF LED
                XGpio_DiscreteWrite(&pmodGpio, CHANNEL, 0);  // Write 0 to Pmod
        }

        usleep(100000); // 100ms debounce

        }

        return 0;
}
```

4.5.3 Results

The program worked as expected. Additionally, the waveform at the Pmod pin was analyzed on a Handheld digital Multimeter which provided an average value, and then on a Digital Oscilloscope. The oscilloscope clearly showed a square waveform, in accordance with the code. The time period and duty cycles of the square waveform were found to be equal to the mentioned value in the bare metal codes.

4.6 AXI GPIO - Polling Implementation

This bare-metal program demonstrates how to implement AXI GPIO-based input and output operations on the ZedBoard using a polling mechanism. The code is written in C and runs directly on the ARM Cortex-A9 Processing System (PS) of the Zynq-7000 SoC using the Xilinx Vitis development environment. The application is built upon the Xilinx-provided hardware abstraction libraries (xgpio.h, xparameters.h, etc.), which facilitate interfacing with hardware IP cores instantiated in the Programmable Logic (PL) via the AXI bus.

4.6.1 Description

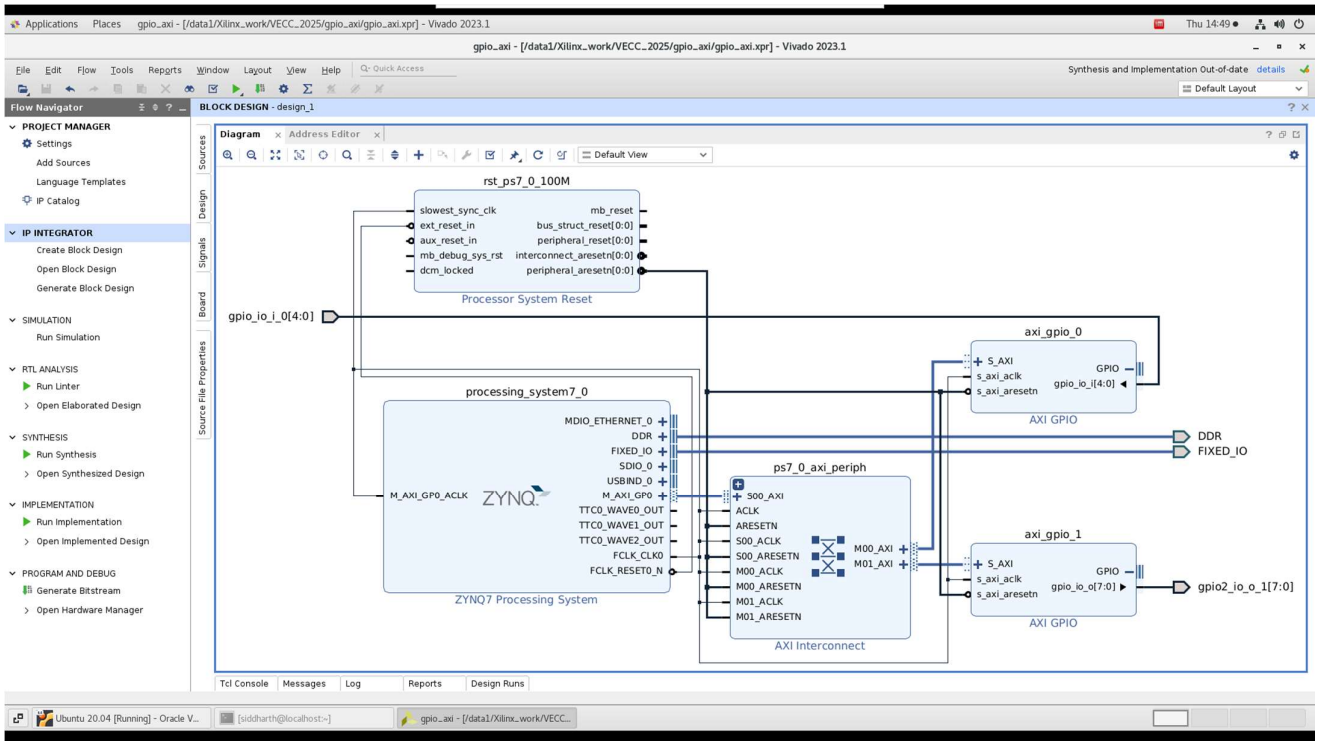
Input: Monitor external input signals (e.g., pushbuttons or switches) connected to an AXI GPIO IP mapped to input channel

Output: Drive LEDs or other output peripherals connected to a second AXI GPIO IP

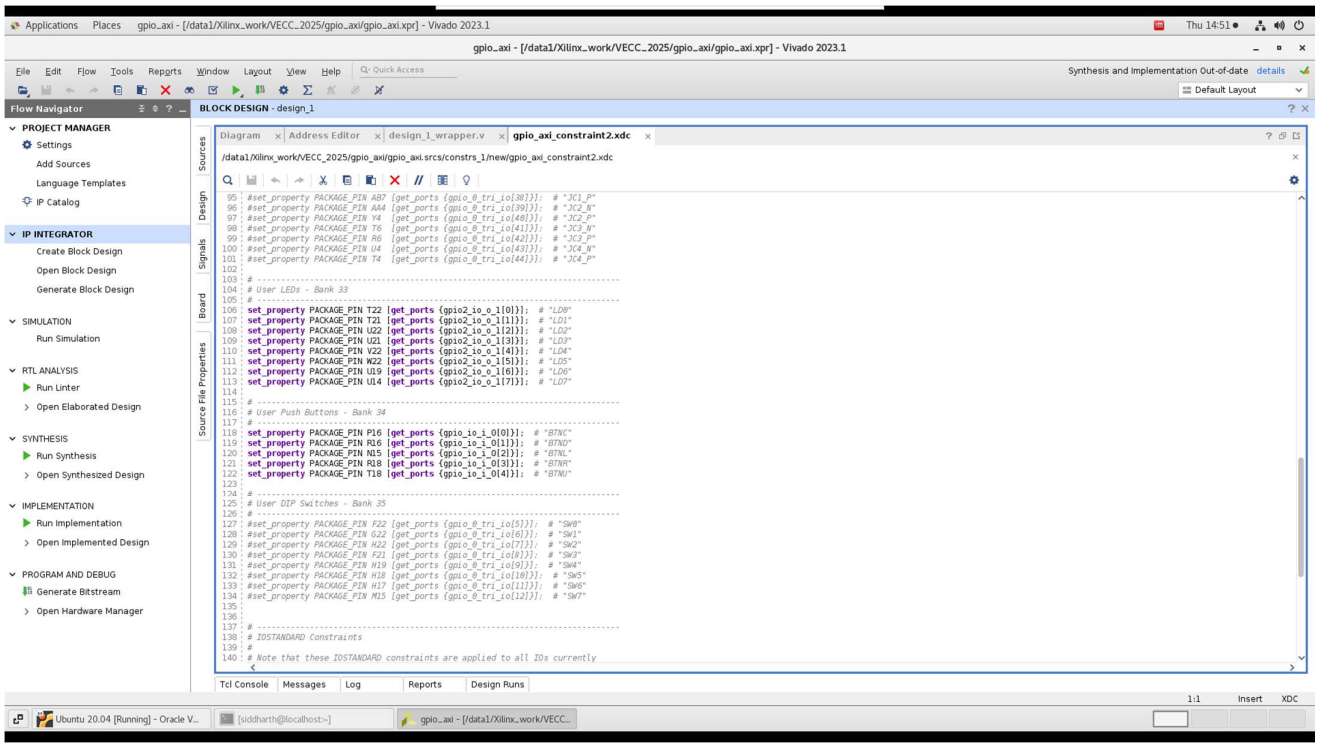
Functionality:

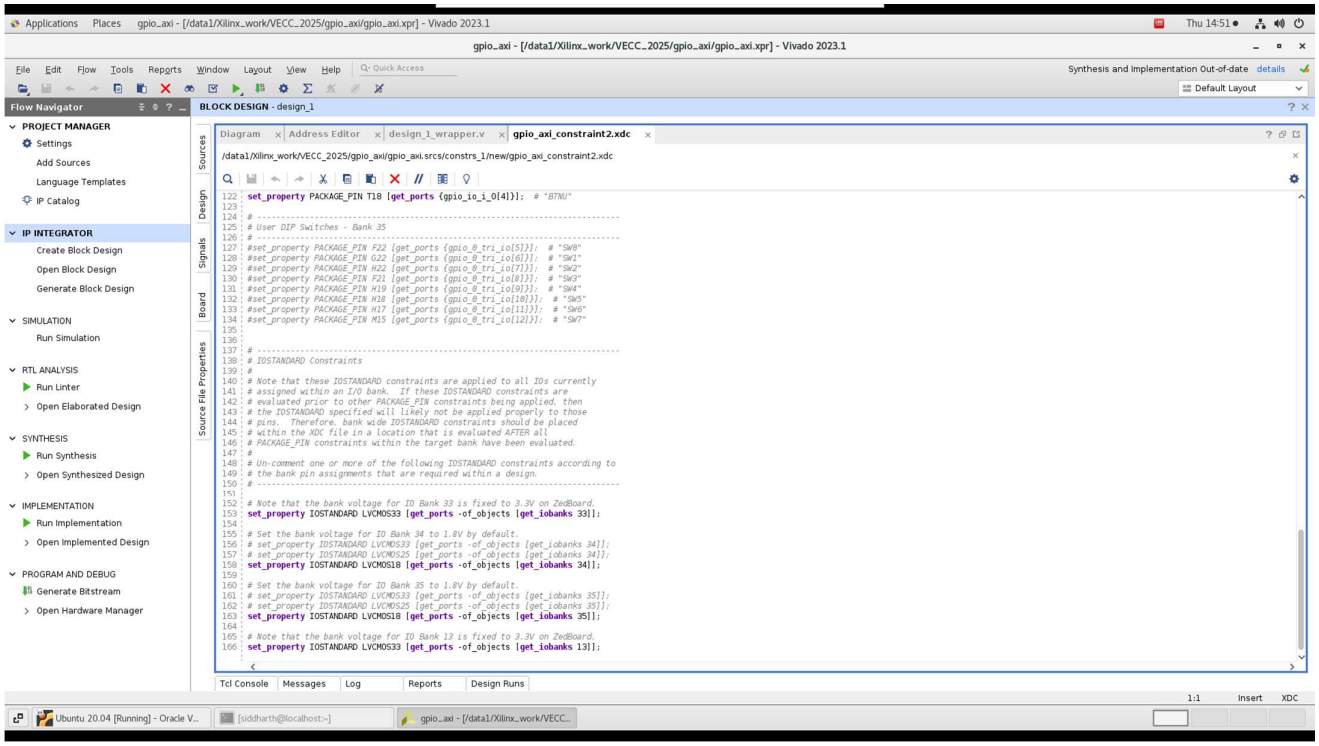
- Print a message to the UART console every second.
- Poll for specific input patterns.
- Based on the input:
    - Write a static value (0x0F) to the output GPIO.
    - Write a sequence of values with 1-second delay to simulate dynamic output.
    - Reset the output to zero if no recognized input is detected.

Vivado Block Diagram:

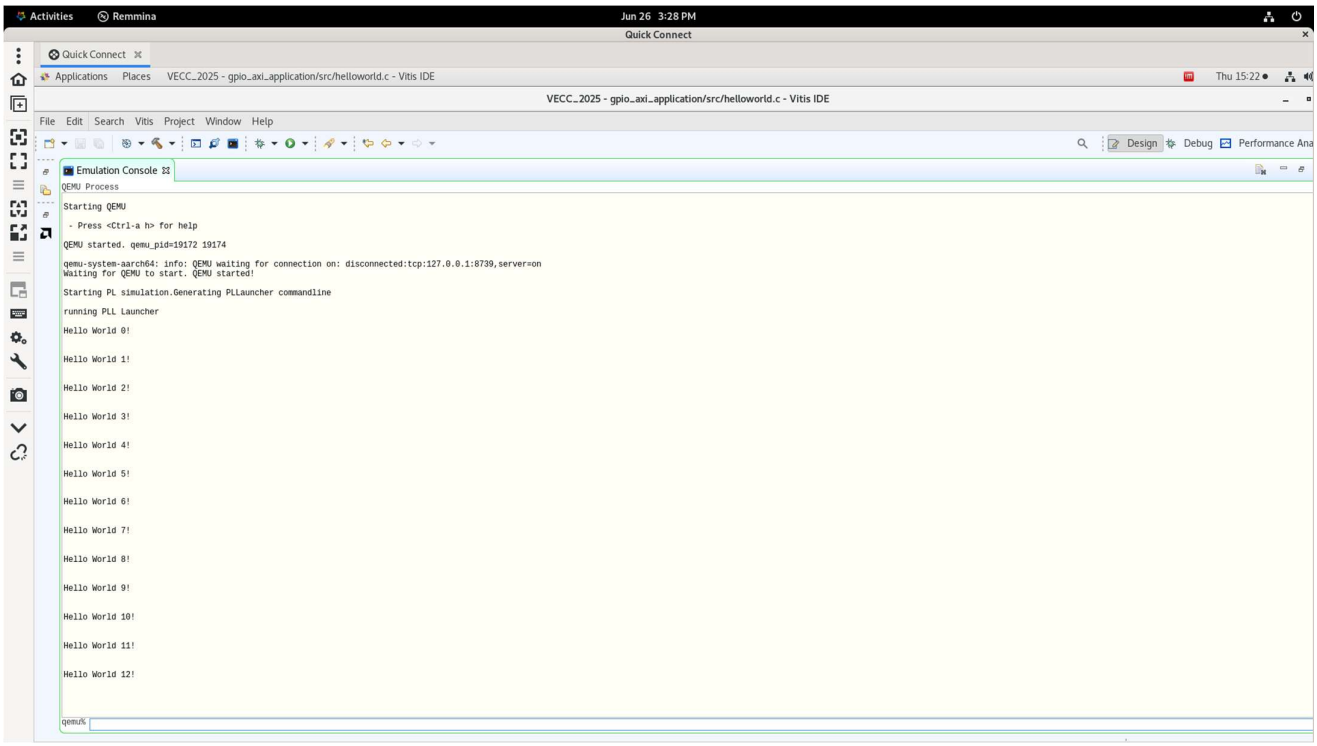


Constraint file:





Vitis Emulator Output:



Here, the AXI GPIO IP core was added to the PL and interfaced with the PS using an AXI GPIO port. The application monitored switch states or button inputs using polling (continuous checking in a loop).

Steps:

- Add and connect AXI GPIO to PS in Vivado
- Map it to switches or buttons on the ZedBoard using .xdc file (constraint file)
- In Vitis, initialize the XGpio driver
- Use XGpio_DiscreteRead() or XGpio_DiscreteWrite() to read/write GPIO values in a loop

4.6.2 Code

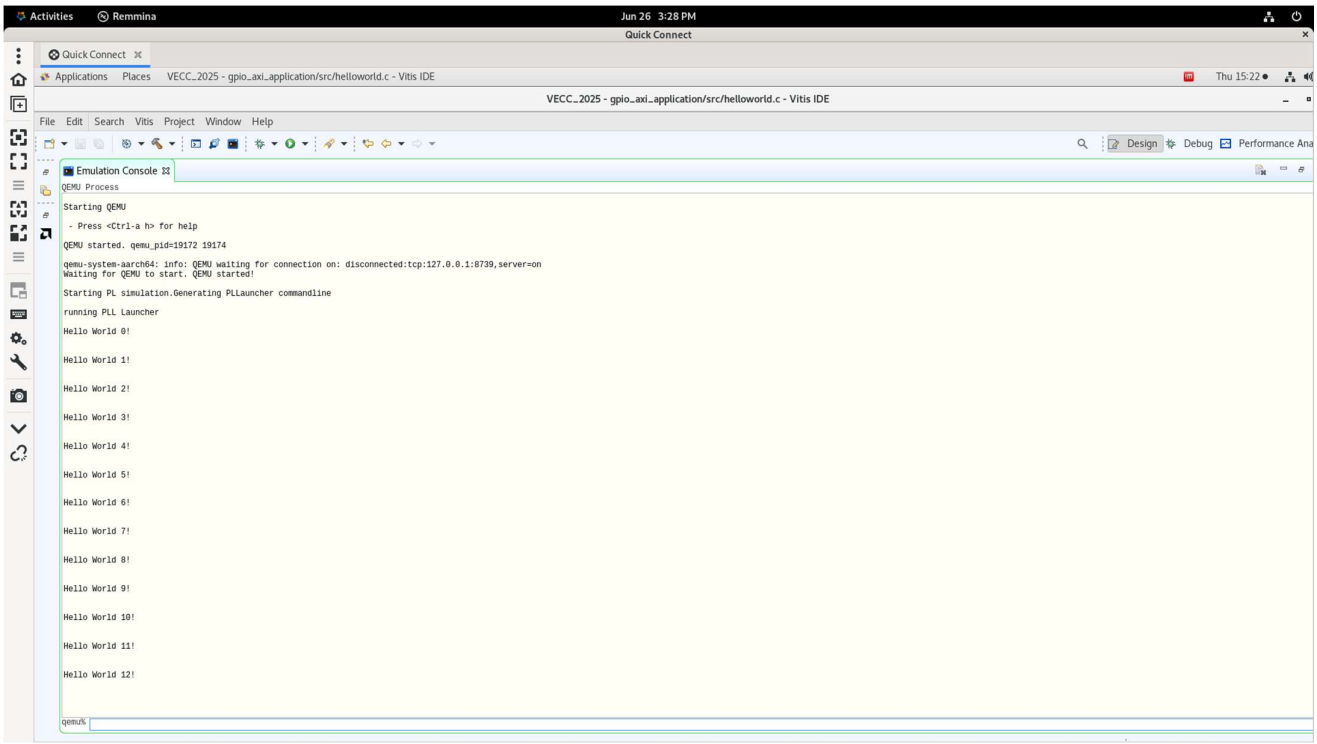
```
#include "xgpio.h"
#include "xparameters.h"
#include "sleep.h"
#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"
#include<time.h>
#include "xgpiops.h"
#include "xstatus.h"
#include "xil_io.h"

XGpio Gpio; /* The driver instance for GPIO Device. */
int main()
{
```

```
int val=0;
XGpio input,output;

XGpio_Initialize(&input, XPAR_AXI_GPIO_0_DEVICE_ID);
XGpio_Initialize(&output, XPAR_AXI_GPIO_1_DEVICE_ID);

XGpio_SetDataDirection(&input,1,1);
XGpio_SetDataDirection(&output,1,0);

int i=0;
while(1){
   printf("Hello World %d!\r\n",i);
   sleep(1);
   i++;

   val=XGpio_DiscreteRead(&input,1);
   //val=Xil_In32(XPAR_AXI_GPIO_0_BASEADDR);

   if(val==0x02)
   {
      XGpio_DiscreteWrite(&output,1,0x0F);
      //Xil_Out32(XPAR_AXI_GPIO_1_BASEADDR,255);

   }

   else if(val==0x04)
   {
      XGpio_DiscreteWrite(&output,1,205);
      //Xil_Out32(XPAR_AXI_GPIO_1_BASEADDR,205);
      sleep(1);
      XGpio_DiscreteWrite(&output,1,127);
      sleep(1);
      XGpio_DiscreteWrite(&output,1,101);
      sleep(1);
      XGpio_DiscreteWrite(&output,1,45);
      sleep(1);
      XGpio_DiscreteWrite(&output,1,190);
      sleep(1);
      XGpio_DiscreteWrite(&output,1,235);
      sleep(1);
      XGpio_DiscreteWrite(&output,1,216);
      sleep(1);
      XGpio_DiscreteWrite(&output,1,169);
      sleep(1);
      XGpio_DiscreteWrite(&output,1,73);
      sleep(1);
```

```
        XGpio_DiscreteWrite(&output,1,67);
        sleep(1);
    }

    else
    {
        XGpio_DiscreteWrite(&output,1,0x00);
        // Xil_Out32(XPAR_AXI_GPIO_1_BASEADDR,0xF0);
     }
}

cleanup_platform();

return 0;

}
```

4.6.3 Results

Vitis Emulator Output:



4.7 AXI GPIO – Interrupt-based Implementation

This section details an interrupt-based GPIO implementation using the AXI GPIO IP and the Xilinx Zynq-7000 SoC. Unlike the polling method, which constantly checks the status of GPIO inputs, this approach configures the system to react asynchronously to hardware events (like switch toggles) via interrupts, making the program more efficient and responsive.

The system utilizes:

- AXI GPIO IP (as input device)
- AXI Interrupt signal routed from PL to PS
- GIC (Generic Interrupt Controller) of the ARM Cortex-A9
- Vitis BSP drivers (xgpio.h, xscugic.h, etc.)

The goal is to toggle a software flag whenever a switch (SW0–SW7) is flipped, using an interrupt instead of continuous polling.

4.7.1 Description

In this implementation, GPIO interrupts were used to trigger an event when a switch state changed. The interrupt was routed from the AXI GPIO IP to the PS via an AXI interrupt controller.

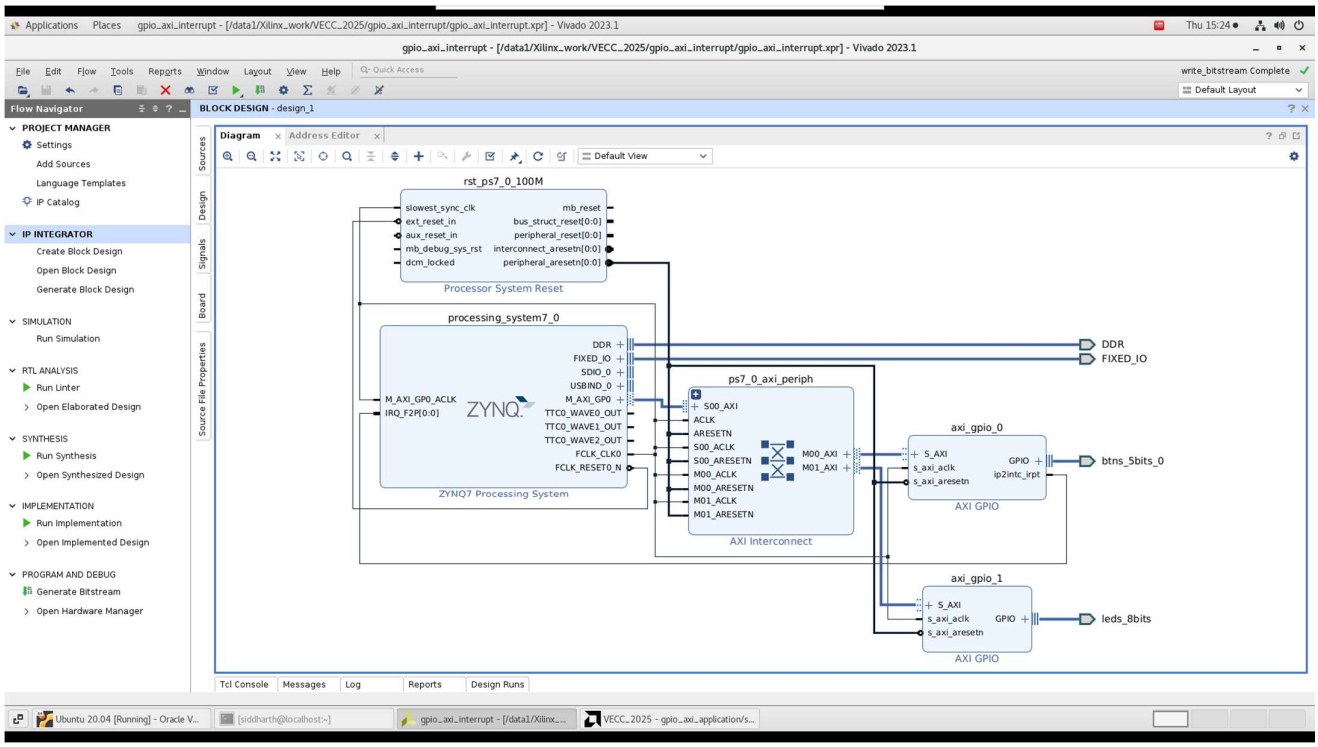
In Vivado:

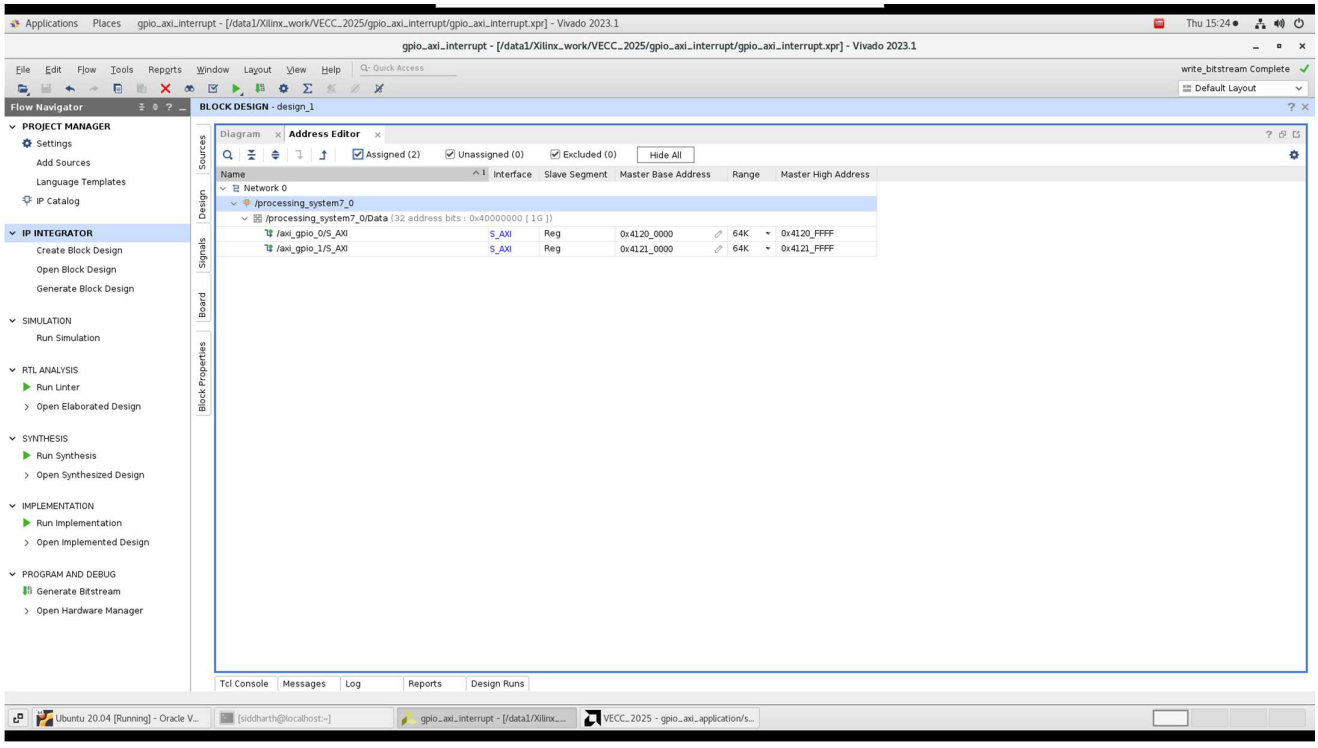
- Add AXI Interrupt Controller and connect it to PS.
- Route interrupt signal from AXI GPIO to the controller.

In Vitis:

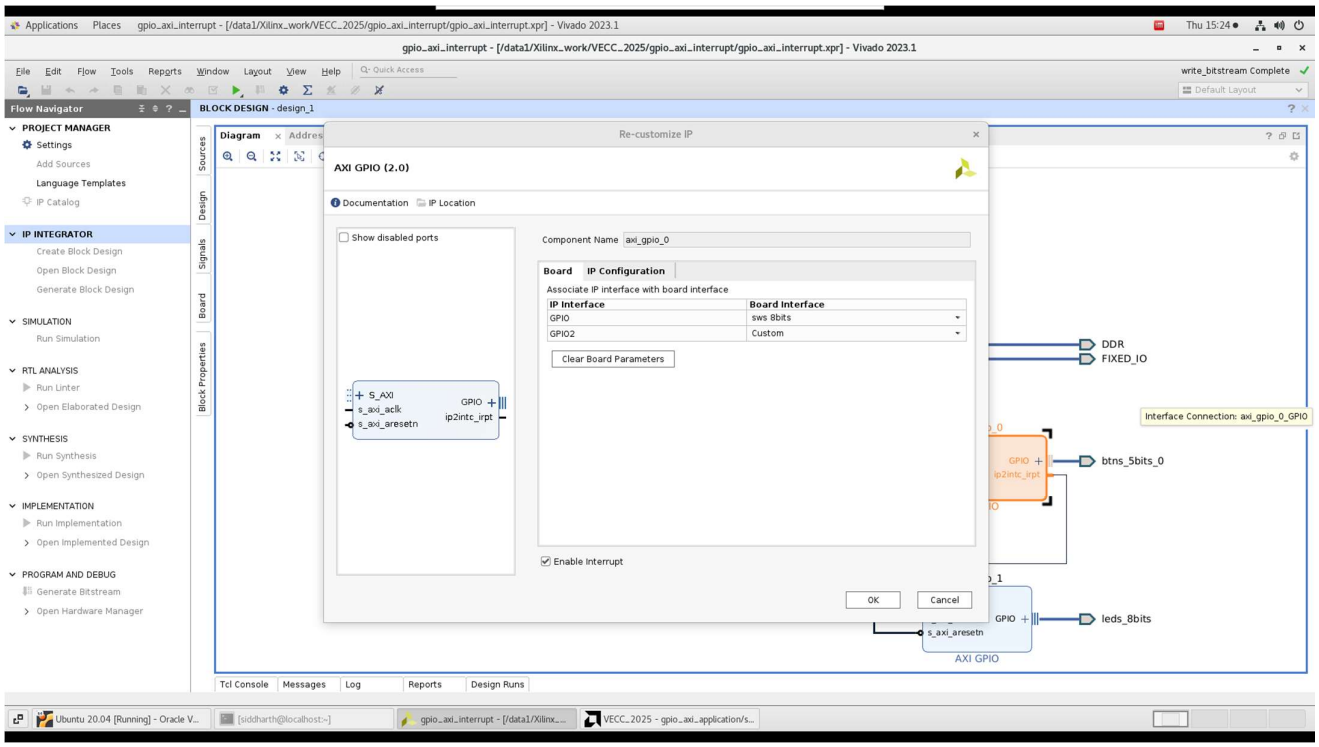
- Register the interrupt handler using XScuGic_Connect().
- Enable global and peripheral-specific interrupts.
- Write an Interrupt Service Routine (ISR) that sets a global flag when the switch is toggled.

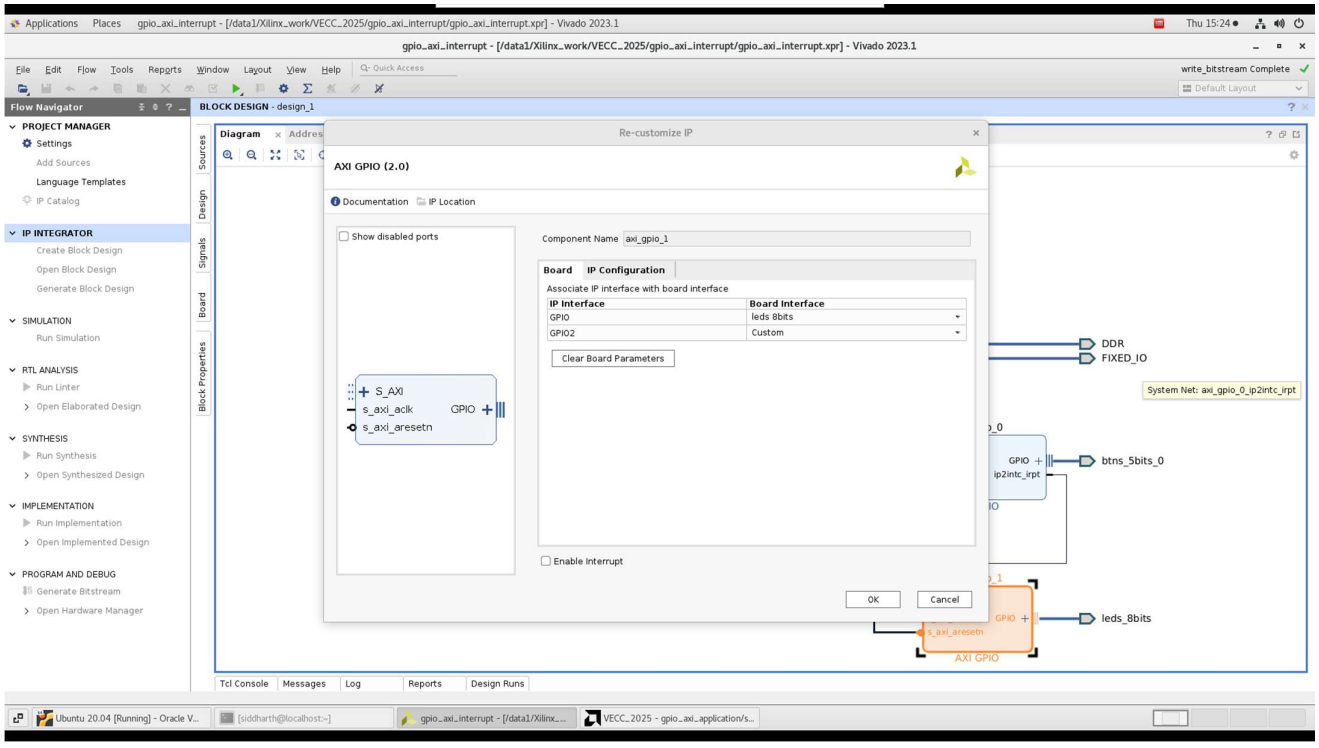
Vivado Block Diagram:

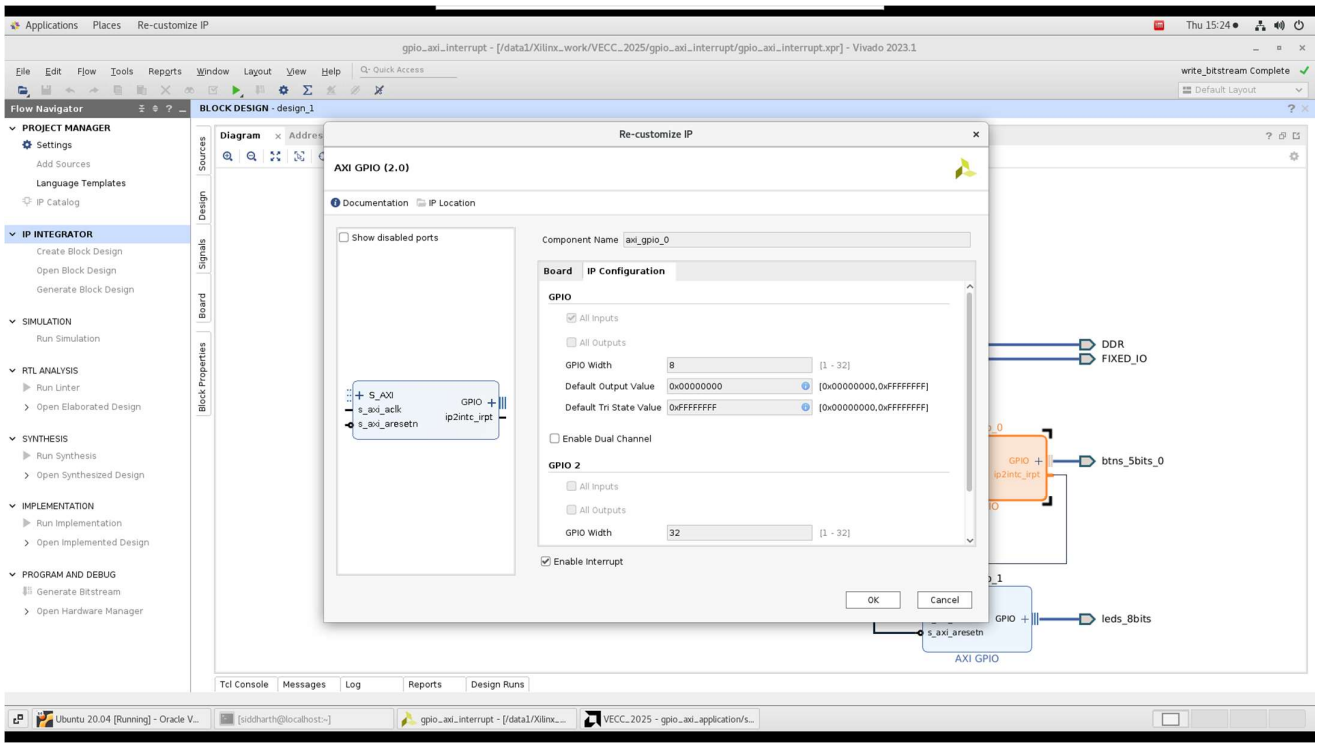


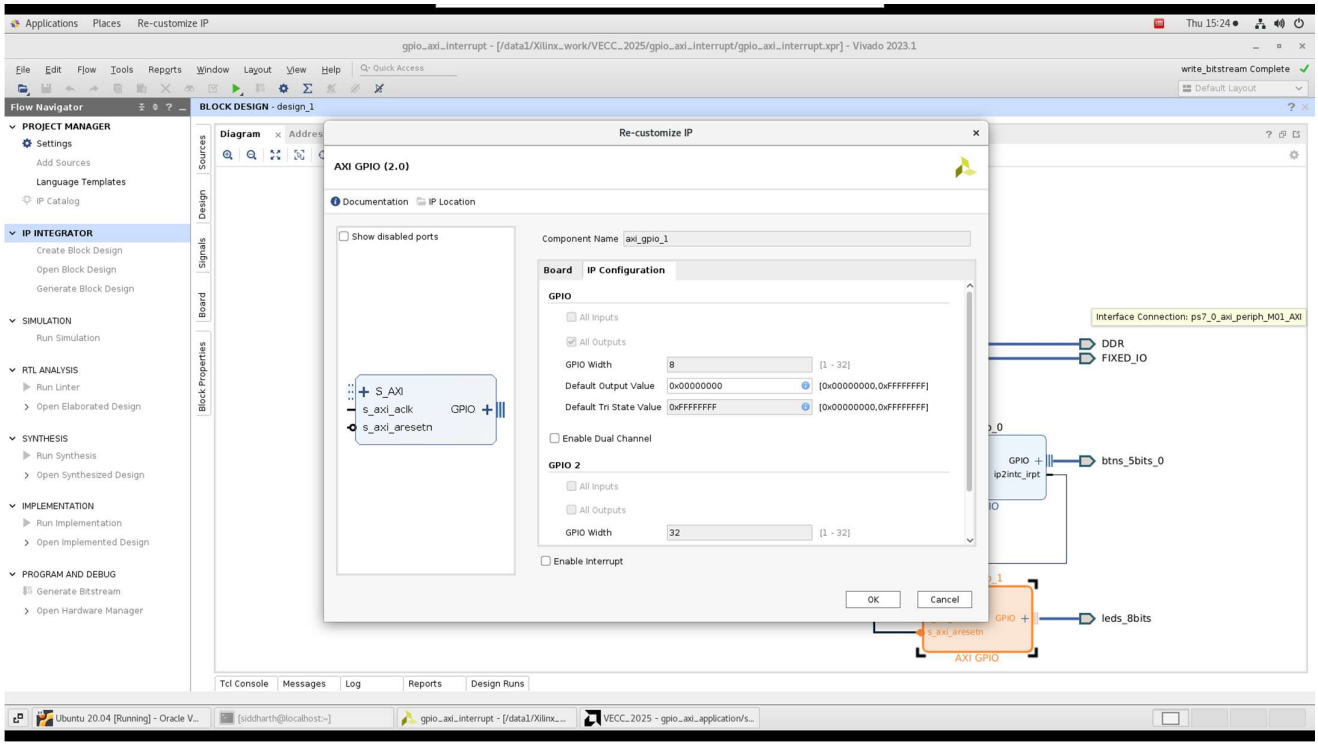


Vivado AXI GPIO Configurations:

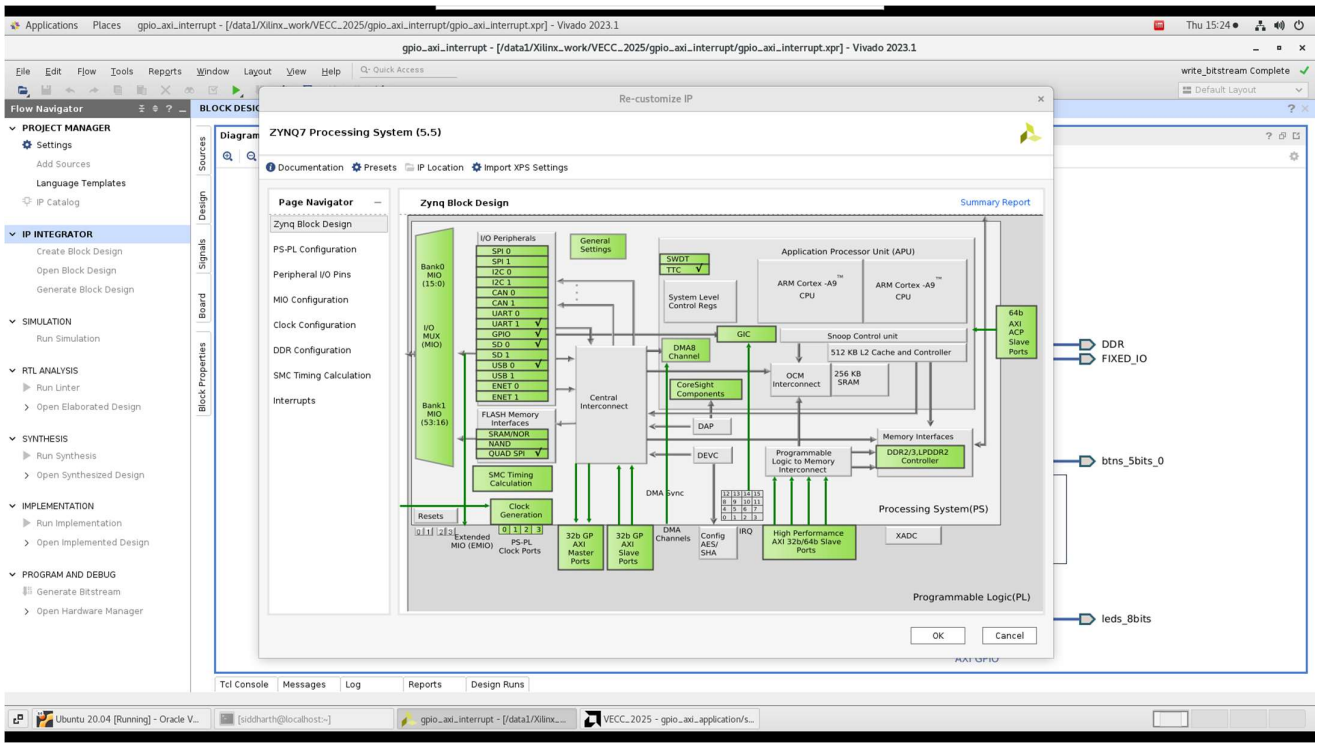

gpio_axi_interrupt - [/data1/Xilinx_work/VECC_2025/gpio_axi_interrupt/gpio_axi_interrupt.xpr] - Vivado 2023.1
Re-customize IP
AXI GPIO (2.0)
Documentation
IP Location
Show disabled ports
Component Name
axi_gpio_0
Board
IP Configuration
Associate IP interface with board interface
IP Interface
Board Interface
GPIO
sws 8bits
GPIO2
Custom
Clear Board Parameters
Enable Interrupt
OK
Cancel
S_AXI
s_axi_aclk
s_axi_aresetn
GPIO
ip2intc_irpt
DDR
FIXED_IO
Interface Connection: axi_gpio_0_GPIO
btns_5bits_0
leds_8bits
AXI GPIO

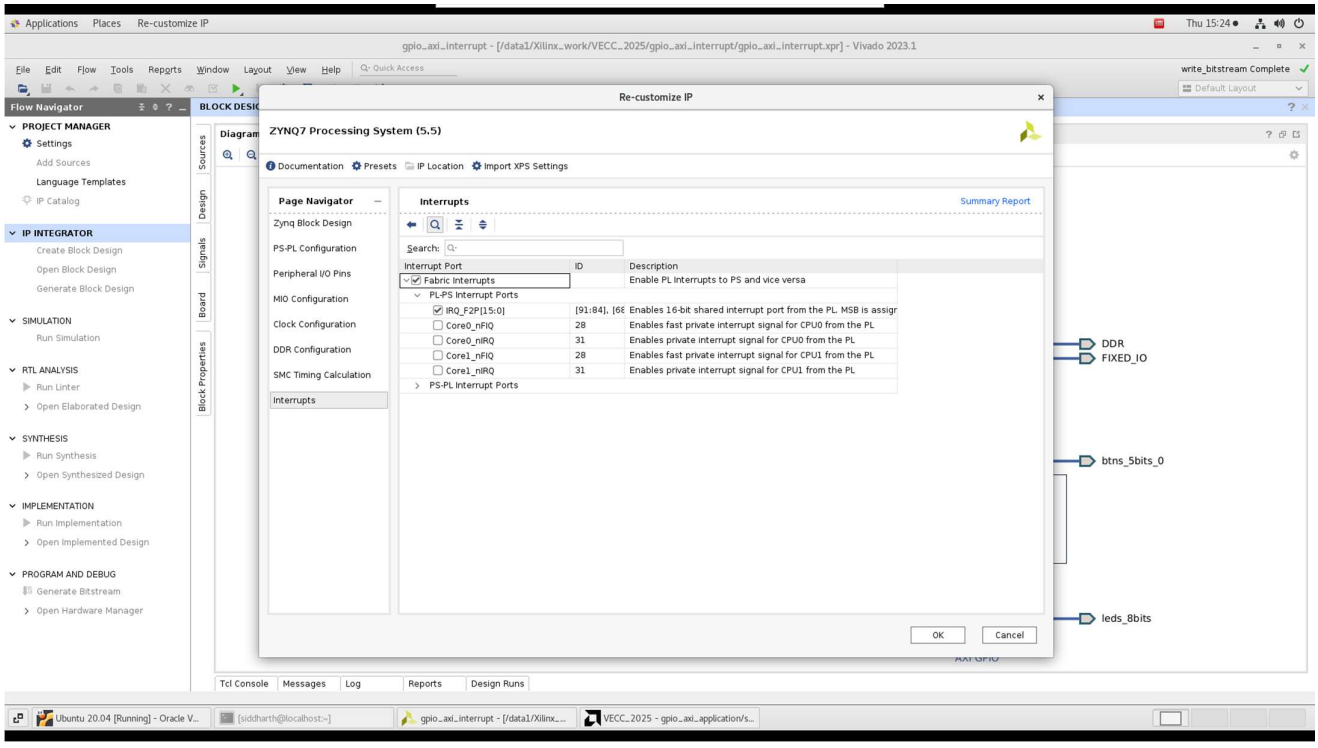

gpio_axi_interrupt - [/data1/Xilinx_work/VECC_2025/gpio_axi_interrupt/gpio_axi_interrupt.xpr] - Vivado 2023.1
Re-customize IP
AXI GPIO (2.0)
Documentation
IP Location
Show disabled ports
Component Name
axi_gpio_1
Board
IP Configuration
Associate IP interface with board interface
IP Interface
Board Interface
GPIO
leds 8bits
GPIO2
Custom
Clear Board Parameters
Enable Interrupt
OK
Cancel
S_AXI
s_axi_aclk
s_axi_aresetn
GPIO
ip2intc_irpt
DDR
FIXED_IO
System Net: axi_gpio_0_ip2intc_irpt
btns_5bits_0
leds_8bits
AXI GPIO





Zynq7 Processing System Configurations:





4.7.2 Code

```
#include "xil_printf.h"
#include "sleep.h"
#include "xgpio.h"
#include "xscugic.h"
#include "xil_exception.h"
#include "xparameters.h"
```

```
#define GPIO_ID XPAR_AXI_GPIO_0_DEVICE_ID
#define GIC_ID XPAR_SCUGIC_SINGLE_DEVICE_ID
#define GPIO_INT_ID XPAR_FABRIC_AXI_GPIO_0_IP2INTC_IRPT_INTR
#define SW_MASK 0xFF /* 8 slide switches: SW0–SW7 */

static XGpio Gpio;
static XScuGic Gic;

volatile int flag=0;

/* -------- Interrupt Service Routine (ISR) -------- */
static void SwitchIsr(void NotUsed)
{
	/* ACK the interrupt first */
	XGpio_InterruptClear(&Gpio, SW_MASK);
	flag = 1;

//xil_printf("switch toggled\r\n ");
}

int main(void)
{
	/* 1. GPIO setup: channel-1 = 8-bit input, enable its IRQ */
	XGpio_Initialize(&Gpio, GPIO_ID);
	XGpio_SetDataDirection(&Gpio, 1, SW_MASK);
	XGpio_InterruptEnable (&Gpio, SW_MASK);
	XGpio_InterruptGlobalEnable(&Gpio);

	/* 2. GIC init + connect ISR */
	XScuGic_Config *Cfg = XScuGic_LookupConfig(GIC_ID);
	XScuGic_CfgInitialize(&Gic, Cfg, Cfg->CpuBaseAddress);
	XScuGic_Connect(&Gic, GPIO_INT_ID,
		(Xil_ExceptionHandler)SwitchIsr, NULL);
	XScuGic_Enable(&Gic, GPIO_INT_ID);

	/* 3. Enable CPU-side interrupts */
	Xil_ExceptionInit();
	Xil_ExceptionRegisterHandler(XIL_EXCEPTION_ID_INT,
			(Xil_ExceptionHandler)XScuGic_InterruptHandler,
			&Gic);
	Xil_ExceptionEnable();

	xil_printf("Ready – toggle SW0…SW7\r\n");

	int j=0;
```

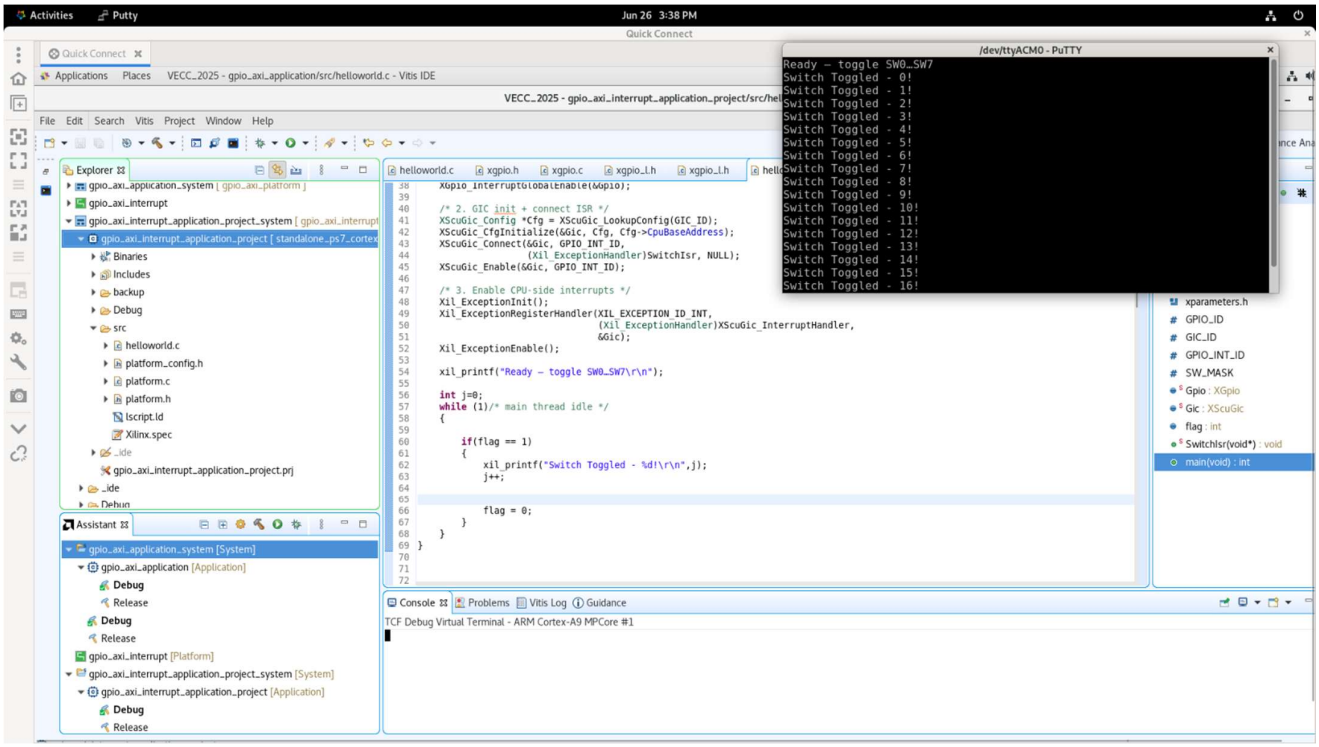
```
        // main thread file
        while (1)
        {

                if(flag == 1)
                {
                        xil_printf("Switch Toggled - %d!\r\n",j);
                        j++;
                        flag=0;


                }
        }

}
```

Reason for using “volatile int” instead of just normal “int”

1. “volatile” keyword tells the compiler that the variable (here flag) may change at any time, outside normal program flow.
2. “int” might cache the value in a register or remove redundant reads while the “volatile” keyword ensures that the compiler must not optimize access to it, every read/write value goes directly to memory.

4.7.3 Results

Putty Serial Terminal Output:



4.7.4 Observations

The polling implementation continuously monitors the GPIO input in an infinite loop, consuming CPU cycles even when no input change occurs. While simple to implement, it is inefficient for real-time or power-sensitive systems. In contrast, the interrupt-based approach configures the system to react only when a switch toggle triggers an interrupt, invoking an ISR that updates a flag checked by the main loop. This results in lower CPU usage, faster responsiveness, and better scalability. Although interrupt setup involves additional configuration, it provides a more efficient and scalable solution for responsive embedded system design, as demonstrated in the above programs.

CHAPTER-V

Linux Porting on Zynq-7000 SoC

This chapter describes the process of porting an embedded Linux distribution, namely PetaLinux, onto the Xilinx Zynq-7000 SoC-based ZedBoard. The goal was to understand setting up an embedded Linux environment tailored to the Zynq architecture. The porting process involved generating a hardware platform in Vivado, configuring and building a PetaLinux project, and finally booting the Linux system on the ZedBoard using an SD card.

5.1 Introduction to Embedded Linux and Petalinux

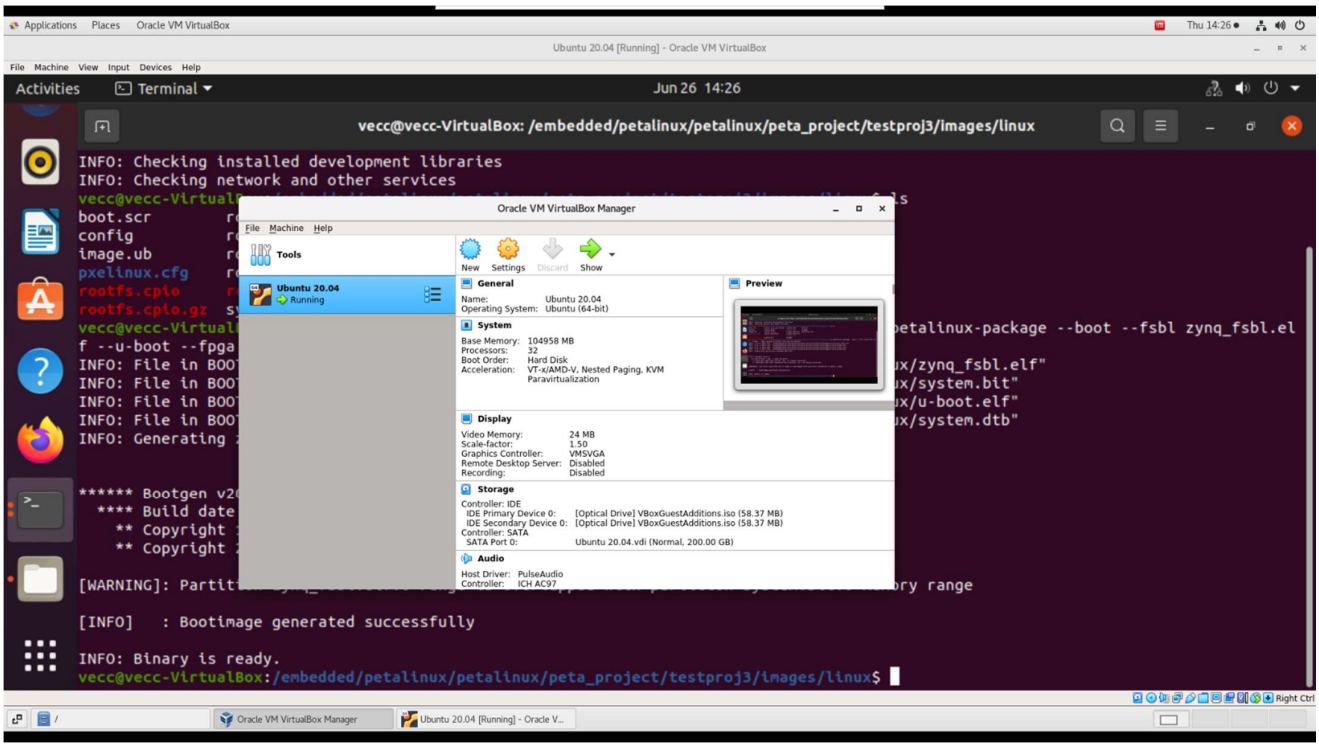
Embedded Linux is a streamlined version of the Linux operating system specifically designed for embedded devices with limited hardware resources. It supports multitasking, networking, file systems, and a vast array of peripheral drivers, making it suitable for a wide range of industrial, automation, and IoT applications.

PetaLinux is Xilinx's official toolchain for creating customized embedded Linux distributions targeting their SoC-based platforms like Zynq-7000. It automates the integration of the hardware platform with Linux components such as the U-Boot bootloader, kernel, device tree, and root filesystem.

5.2 Petalinux Installation

PetaLinux was installed on the virtualized Ubuntu environment. The installation steps are as follows:

1. Virtual Machine Setup:



- Ubuntu 20.04 LTS was installed in Oracle VirtualBox
- Adequate system resources were allocated

2. Installing Dependencies:

- Essential packages such as gcc, libssl-dev, make, libncurses5-dev, and others were installed

3. PetaLinux Installation:

- Downloaded PetaLinux *.run* file from Xilinx's official portal
- Installed to desired directory using:

./<petalinux.run file> --dir <path to directory for installation>

Issues that may arise during this section are detailed in Appendix B.1.

4. Environment Setup:

- Activated via:

source <path to settings.sh file>

5.3 Hardware Description Import from Vivado

The aforementioned Vivado workflow was followed using Zedboard as the target. The Zynq PS IP was instantiated and configures to enable interfaces such as UART, GPIO, SD Card, and DDR. The block design was validated, synthesized and exported with bitstream. The exported hardware specification .xsa file was used as input to PetaLinux.

5.4 Creating and Configuring Petalinux Project

Using the .xsa file from Vivado, a new PetaLinux project was created

```
petalinux-create --type project --template zynq --name <name of project>
cd <name of project>
petalinux-config --get-hw-description <path to .xsa file>
```

This initialised a full project with kernel configuration, U-Boot settings, and device tree templates matched to the hardware

5.5 [Optional] Customizing Device Tree and Root Filesystem

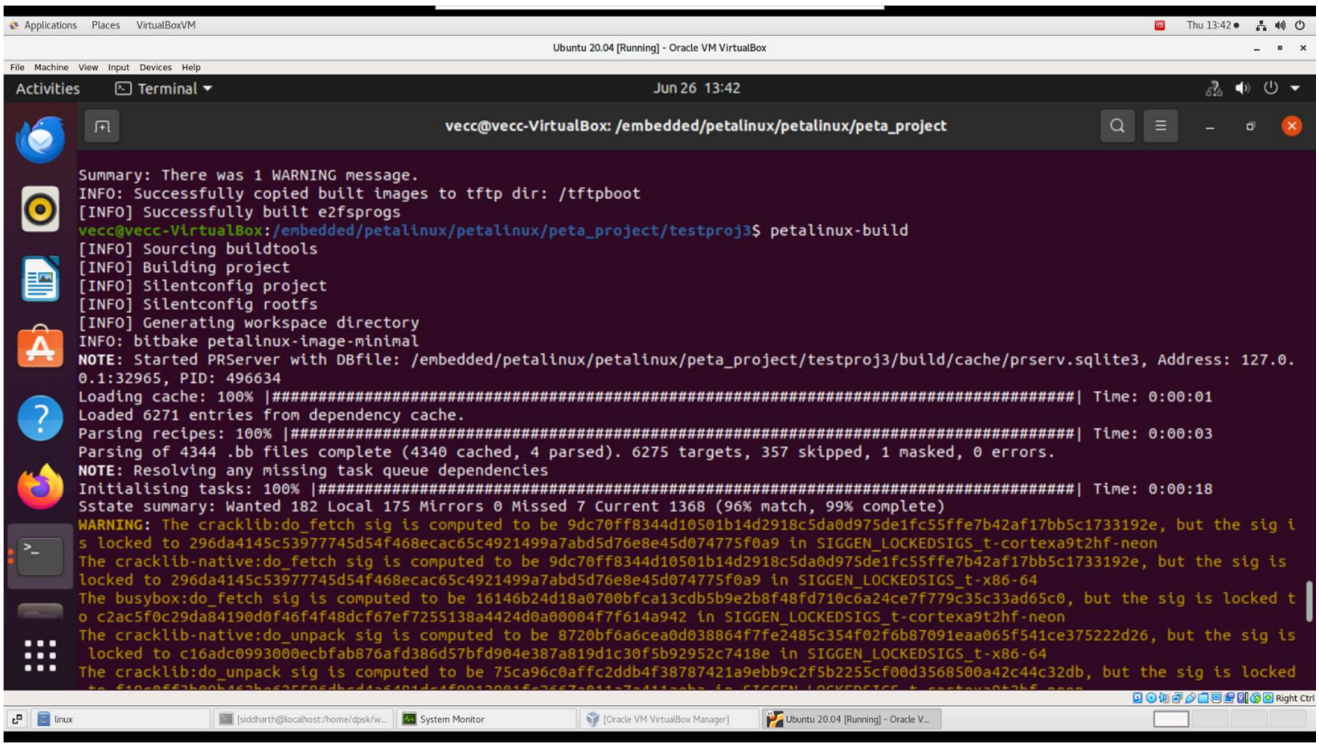
For extended functionality:

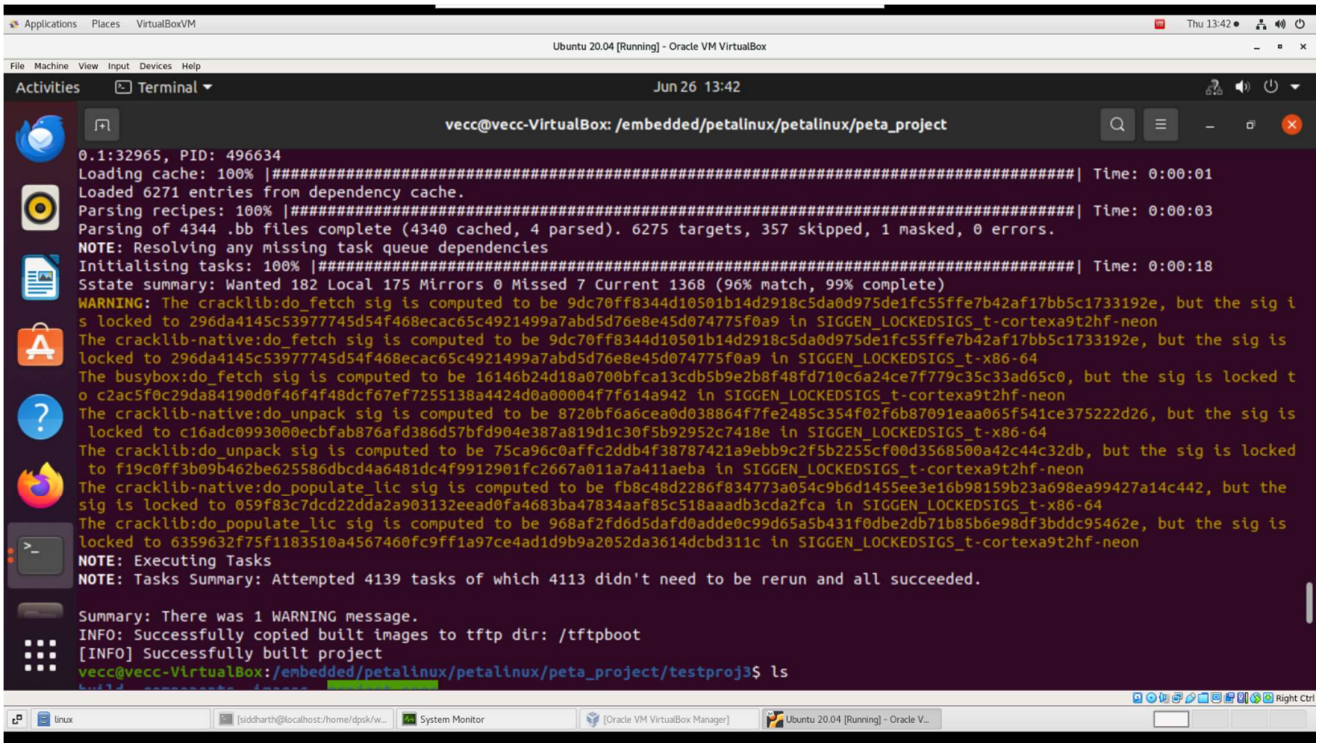
- Custom nodes could be added to system-user.dtsi for custom peripherals or GPIOs.
- Additional packages (e.g., SSH server, GPIO utilities) could be enabled via:

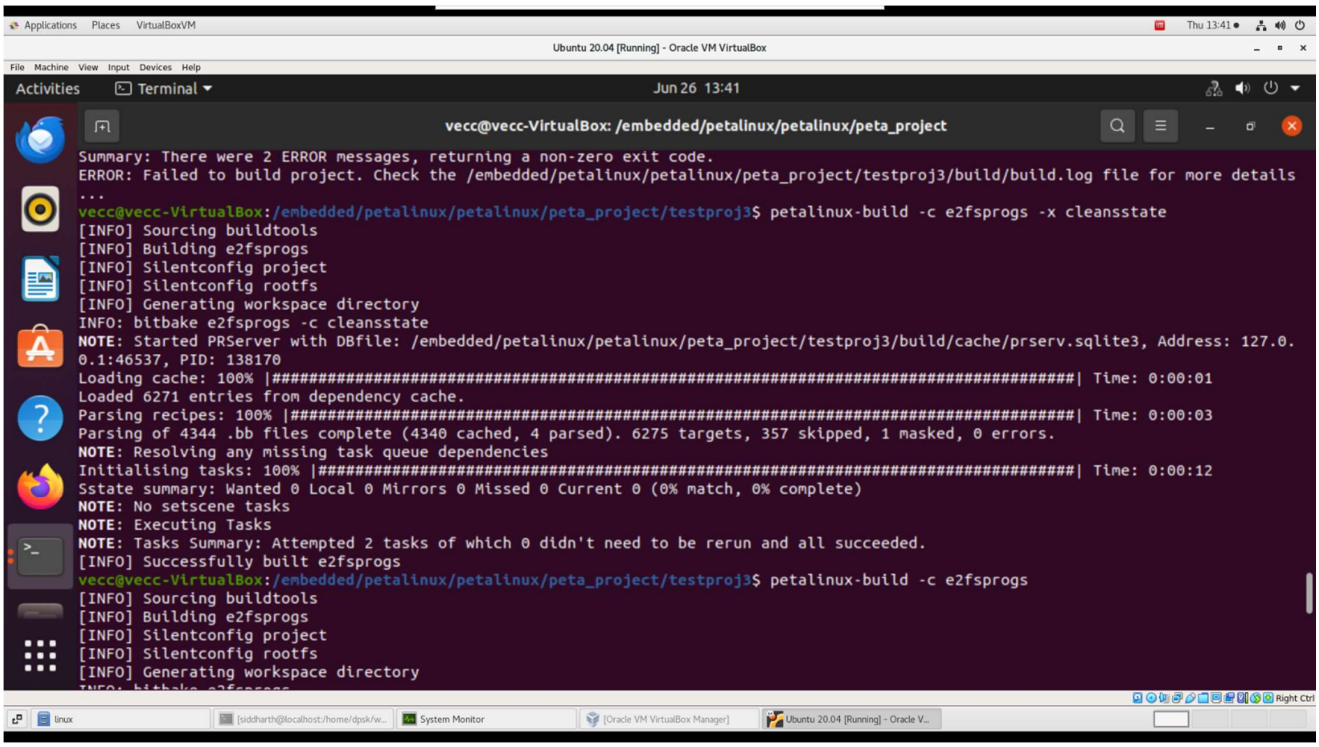
```
petalinux-config -c rootfs
```

However, for this project, the default configuration was sufficient to validate booting and basic hardware interaction.

5.6 Building and Packaging Petalinux


Applications Places VirtualBoxVM
Thu 13:42
Ubuntu 20.04 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Activities Terminal Jun 26 13:42
vecc@vecc-VirtualBox: /embedded/petalinux/petalinux/peta_project
Summary: There was 1 WARNING message.
INFO: Successfully copied built images to tftp dir: /tftpboot
[INFO] Successfully built e2fsprogs
vecc@vecc-VirtualBox:/embedded/petalinux/petalinux/peta_project/testproj3$ petalinux-build
[INFO] Sourcing buildtools
[INFO] Building project
[INFO] Silentconfig project
[INFO] Silentconfig rootfs
[INFO] Generating workspace directory
INFO: bitbake petalinux-image-minimal
NOTE: Started PRServer with DBfile: /embedded/petalinux/petalinux/peta_project/testproj3/build/cache/prserv.sqlite3, Address: 127.0.0.1:32965, PID: 496634
Loading cache: 100% || Time: 0:00:01
Loaded 6271 entries from dependency cache.
Parsing recipes: 100% || Time: 0:00:03
Parsing of 4344 .bb files complete (4340 cached, 4 parsed). 6275 targets, 357 skipped, 1 masked, 0 errors.
NOTE: Resolving any missing task queue dependencies
Initialising tasks: 100% || Time: 0:00:18
Sstate summary: Wanted 182 Local 175 Mirrors 0 Missed 7 Current 1368 (96% match, 99% complete)
WARNING: The cracklib:do_fetch sig is computed to be 9dc70ff8344d10501b14d2918c5da0d975de1fc55ffe7b42af17bb5c1733192e, but the sig is locked to 296da4145c53977745d54f468ecac65c4921499a7abd5d76e8e45d074775f0a9 in SIGGEN_LOCKEDSIGS_t-cortexa9t2hf-neon
The cracklib-native:do_fetch sig is computed to be 9dc70ff8344d10501b14d2918c5da0d975de1fc55ffe7b42af17bb5c1733192e, but the sig is locked to 296da4145c53977745d54f468ecac65c4921499a7abd5d76e8e45d074775f0a9 in SIGGEN_LOCKEDSIGS_t-x86-64
The busybox:do_fetch sig is computed to be 16146b24d18a0700bfca13cdb5b9e2b8f48fd710c6a24ce7f779c35c33ad65c0, but the sig is locked to c2ac5f0c29da84190d0f46f4f48dcf67ef7255138a4424d0a00004f7f614a942 in SIGGEN_LOCKEDSIGS_t-cortexa9t2hf-neon
The cracklib-native:do_unpack sig is computed to be 8720bf6a6cea0d038864f7fe2485c354f02f6b87091eaa065f541ce375222d26, but the sig is locked to c16adc0993000ecbfab876afd386d57bfd904e387a819d1c30f5b92952c7418e in SIGGEN_LOCKEDSIGS_t-x86-64
The cracklib:do_unpack sig is computed to be 75ca96c0affc2ddb4f38787421a9ebb9c2f5b2255cf00d3568500a42c44c32db, but the sig is locked
Right Ctrl
linux
[siddharth@localhost:/home/dpsk/w...
System Monitor
[Oracle VM VirtualBox Manager]
Ubuntu 20.04 [Running] - Oracle V...


Applications Places VirtualBoxVM
Thu 13:42
Ubuntu 20.04 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Activities Terminal Jun 26 13:42
vecc@vecc-VirtualBox: /embedded/petalinux/petalinux/peta_project
0.1:32965, PID: 496634
Loading cache: 100% || Time: 0:00:01
Loaded 6271 entries from dependency cache.
Parsing recipes: 100% || Time: 0:00:03
Parsing of 4344 .bb files complete (4340 cached, 4 parsed). 6275 targets, 357 skipped, 1 masked, 0 errors.
NOTE: Resolving any missing task queue dependencies
Initialising tasks: 100% || Time: 0:00:18
Sstate summary: Wanted 182 Local 175 Mirrors 0 Missed 7 Current 1368 (96% match, 99% complete)
WARNING: The cracklib:do_fetch sig is computed to be 9dc70ff8344d10501b14d2918c5da0d975de1fc55ffe7b42af17bb5c1733192e, but the sig is locked to 296da4145c53977745d54f468ecac65c4921499a7abd5d76e8e45d074775f0a9 in SIGGEN_LOCKEDSIGS_t-cortexa9t2hf-neon
The cracklib-native:do_fetch sig is computed to be 9dc70ff8344d10501b14d2918c5da0d975de1fc55ffe7b42af17bb5c1733192e, but the sig is locked to 296da4145c53977745d54f468ecac65c4921499a7abd5d76e8e45d074775f0a9 in SIGGEN_LOCKEDSIGS_t-x86-64
The busybox:do_fetch sig is computed to be 16146b24d18a0700bfca13cdb5b9e2b8f48fd710c6a24ce7f779c35c33ad65c0, but the sig is locked to c2ac5f0c29da84190d0f46f4f48dcf67ef7255138a4424d0a00004f7f614a942 in SIGGEN_LOCKEDSIGS_t-cortexa9t2hf-neon
The cracklib-native:do_unpack sig is computed to be 8720bf6a6cea0d038864f7fe2485c354f02f6b87091eaa065f541ce375222d26, but the sig is locked to c16adc0993000ecbfab876afd386d57bfd904e387a819d1c30f5b92952c7418e in SIGGEN_LOCKEDSIGS_t-x86-64
The cracklib:do_unpack sig is computed to be 75ca96c0affc2ddb4f38787421a9ebb9c2f5b2255cf00d3568500a42c44c32db, but the sig is locked to f19c0ff3b09b462be625586dbcd4a6481dc4f9912901fc2667a011a7a411aeba in SIGGEN_LOCKEDSIGS_t-cortexa9t2hf-neon
The cracklib-native:do_populate_lic sig is computed to be fb8c48d2286f834773a054c9b6d1455ee3e16b98159b23a698ea99427a14c442, but the sig is locked to 059f83c7dcd22dda2a903132eead0fa4683ba47834aaf85c518aaadb3cda2fca in SIGGEN_LOCKEDSIGS_t-x86-64
The cracklib:do_populate_lic sig is computed to be 968af2fd6d5dafd0adde0c99d65a5b431f0dbe2db71b85b6e98df3bddc95462e, but the sig is locked to 6359632f75f1183510a4567460fc9ff1a97ce4ad1d9b9a2052da3614dcbd311c in SIGGEN_LOCKEDSIGS_t-cortexa9t2hf-neon
NOTE: Executing Tasks
NOTE: Tasks Summary: Attempted 4139 tasks of which 4113 didn't need to be rerun and all succeeded.
Summary: There was 1 WARNING message.
INFO: Successfully copied built images to tftp dir: /tftpboot
[INFO] Successfully built project
vecc@vecc-VirtualBox:/embedded/petalinux/petalinux/peta_project/testproj3$ ls
Right Ctrl
linux
[siddharth@localhost:/home/dpsk/w...
System Monitor
[Oracle VM VirtualBox Manager]
Ubuntu 20.04 [Running] - Oracle V...



The full build process was initiated with:

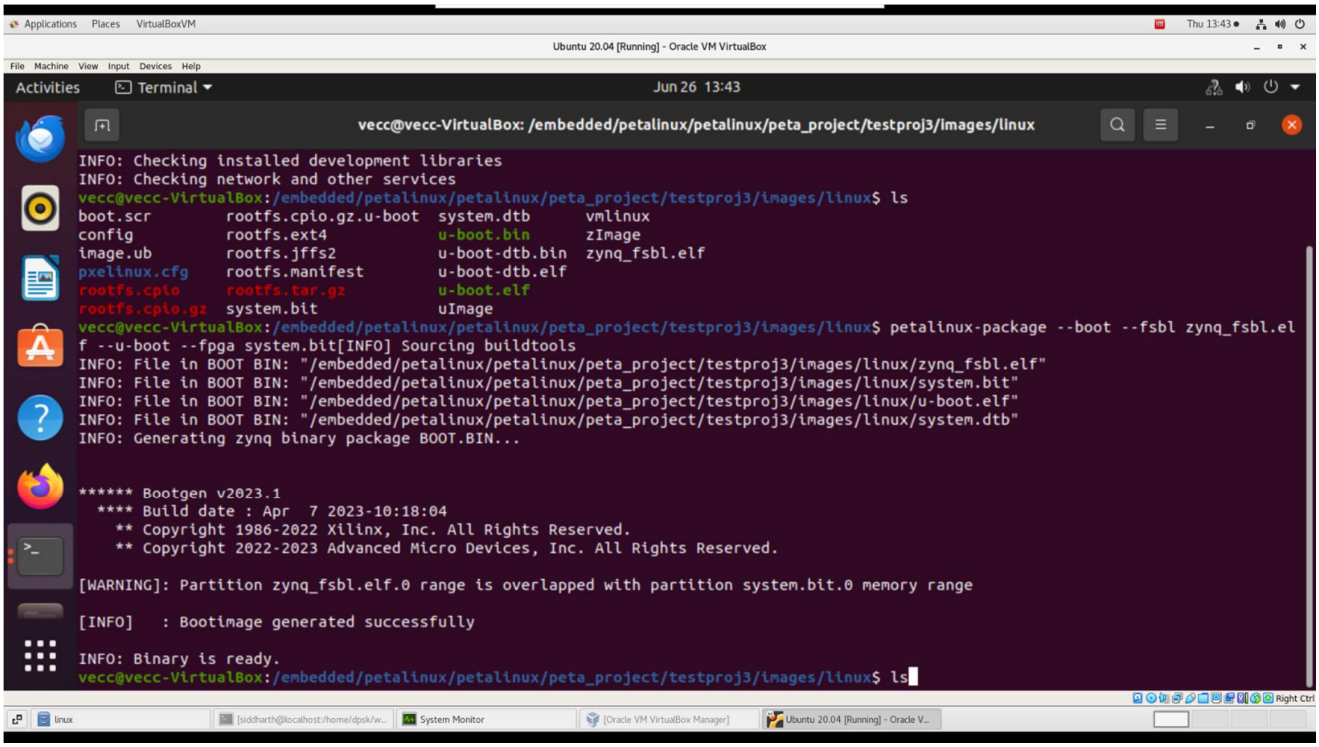
petalinux-build

Issues that may arise during this section are detailed in Appendix B.2.

This step generated:

- BOOT.BIN: First-stage bootloader + bitstream + U-Boot
- Image.ub: Linux kernel, device tree, and root filesystem

For packaging:



petalinux-package --boot --fsbl zynq_fsbl.elf --u-boot --fpga system.bit

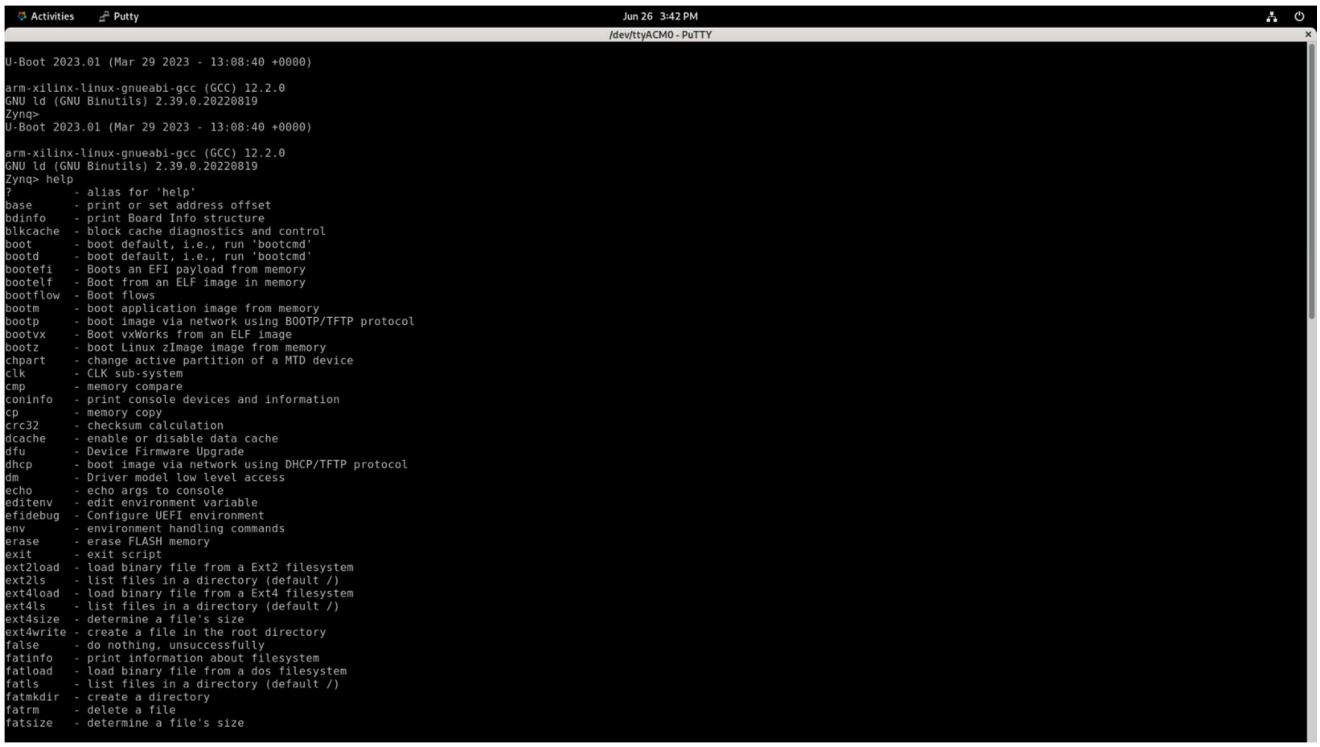
5.7 Sd Card Preparation and Booting on Zedboard

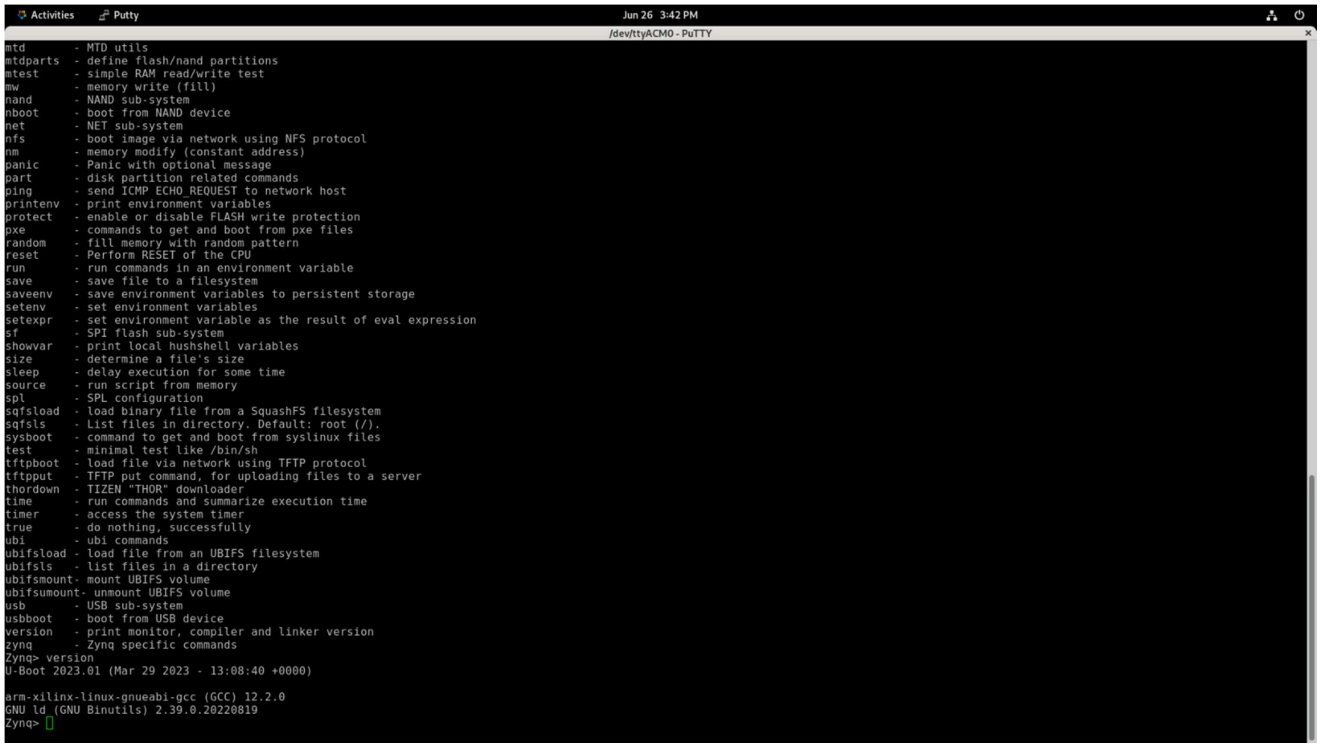
1. The SD card was formatted in (FAT32 filesystem).
2. BOOT.BIN and image.ub were copied to the SD card.
3. The SD card was inserted into the ZedBoard.
4. Boot mode DIP switched were configured for SD boot.
5. On power-up, the board executed the FSBL (First Stage Bootloader) and launched Linux from the SD card.

5.8 Debugging and Serial Console Verification

1. The ZedBoard was connected to the host via a USB-UART interface.
2. Serial communication was set up using PuTTY at 115200 baud, 8N1.
3. Upon successful boot, console logs appeared showing the U-Boot loader, kernel boot sequence, and login prompt

Output:





This confirmed that PetaLinux was successfully ported and operational.

CHAPTER-VI

Conclusion

6.1 Summary of Work Done

During this internship at the Variable Energy Cyclotron Centre (VECC), We gained hands-on experience with the Zynq-7000 SoC-based ZedBoard and Zybo Z7 through a combination of bare-metal and embedded Linux development workflows. The internship began with a deep dive into the Zynq architecture, including the Processing System (PS), Programmable Logic (PL), and their interfacing mechanisms. I implemented several bare-metal applications using Xilinx Vivado and Vitis, such as UART-based "Hello World" output and GPIO control via MIO, EMIO, and AXI interfaces using both polling and interrupt-driven techniques. The second phase of the internship involved porting PetaLinux onto the ZedBoard. This included creating a virtual environment with Ubuntu 20.04, installing and configuring the PetaLinux SDK, importing the hardware description from Vivado, building and packaging the Linux image, and successfully booting it on the ZedBoard via SD card.

6.2 Future Scope

The work carried out in this project lays a solid foundation for advanced embedded system development using the Zynq-7000 SoC platform. In the future, this setup can be extended to a custom-built development board which would be catered to the needs of the LLRF control system. This custom-built development board, with only a few required peripherals, could be used for implementation of real-time control systems or high-speed data acquisition pipelines using parallel implementation of required logic. The interrupt-driven GPIO framework can be enhanced to support multiple peripherals and advanced event-handling mechanisms.

On the software side, the PetaLinux can further be ported to this custom-built development board. For this to happen, softwares such as – device tree, bootloader, Linux kernel and the Linux rootfs, would have to be written from scratch (or existing softwares for standard boards can be customized according to our needs). Additionally, the PetaLinux environment can be customized further to include networking stacks, remote debugging tools, or lightweight web servers for embedded monitoring. Moreover, the platform can be adapted for use in domains like LLRF control systems, instrumentation or accelerator subsystem automation at VECC.

CHAPTER-VII

Appendix

A. Problems related to Bare-Metal Programming

A.1 Even after running Vitis application project, nothing was coming up on serial terminal

- UART0 or UART1 may be getting mapped to EMIO peripheral.
- Check Zynq7 Processing System configuration

A.2 Board is showing as connected in Vivado in remote server, but there was an error in Vitis. Some error like "Could not find ARM device on connection: Local" is coming when trying to launch the application project on the hardware (ZedBoard)

- Create a new configuration for the launch and incorporated the IP address & port number (of connection created by hw_server running on local computer) in the configuration.

B. Problems related to PetaLinux

B.1 Permissions errors may come when trying to install PetaLinux

1. Create a directory for petalinux installation as a non-root user
2. Log in as root user and execute the following commands
   - sudo chmod +x ./<file.run>
   - sudo chmod –R 777 <path to created directory>
3. Log in as non-root user and execute this command
   - ./<file.run> --dir <path to .run file> <path to just created directory>

B.2 Many errors may come during petalinux-build, the problems and their possible solutions are listed below:

1. [Warning] No recipes in default available for: /<path to petalinux project>/project-spec/meta-user/recipes-core/images/petalinux_image.bbappend

- mv petalinux_image.bbappend petalinux_image-minimal.bbappend

2. Error related to Cracklib (like "not able to fetch a specific commit <d9e8f9...> from the cracklib GitHub repository's master branch")

- Go to <project name>/components/yocto/layers/poky/meta/recipies-extended/cracklib/cracklib_2.9.8.bb file
- And check for SRC_URI line and in the SRC_URI change branch from master to main (depending on the actual GitHub repository)

3. Locked-sigs.inc file warnings

- When doing petalinux-build, a lot of warnings came regarding signature mismatch and locked-sigs.inc file.
- Sources say to delete the file but if this is done then the project cannot be built (petalinux-build will show errors)
- Solution: If you accidently deleted it then create a file with same name at that path again (nothing to write in the file, just create a file with the same name)
- Don't Do: Go to <project_name>/build/conf/local.conf and comment or uncomment line "require conf/locked-signs.inc" -- doesn't help fix this issue

4. If petalinux-build gets stopped in between or corrupted or showing too many errors:

- Try these commands:
  - petalinux-build –c e2fsprogs –x cleansstate
  - petalinux-build –c e2fsprogs

C. Problems related to Virtual Machine:

C.1 Ubuntu is running on Oracle VM, but network issues are there.

- Then log in as root or non-root user and edit the file /etc/apt/sources.list and add these 4 lines at the end:

deb https://archive.ubuntu.com/ubuntu focal main restricted universe multiverse
deb https://archive.ubuntu.com/ubuntu focal-updates main restricted universe multiverse
deb https://archive.ubuntu.com/ubuntu focal-backports main restricted universe multiverse
deb https://archive.ubuntu.com/ubuntu focal-security main restricted universe multiverse